



Kubernetes on OCI: OKE Cluster Provisioning, Modular Terraform, and a Stateful Application Deployment

Ali Yousef

Week 3 Documentation

Compartment: intern-02-ali-youssef-cmp

Region: me-jeddah-1 (Saudi Arabia West – Jeddah)

Summary

In this documentation, I cover a **managed Kubernetes environment** I built on Oracle Cloud Infrastructure (OCI) Kubernetes Engine (OKE), provisioned with modular Terraform and carrying a stateful application deployed on top of it with `kubect1`. Weeks 1 and 2 of this internship stayed inside OCI's own networking and compute primitives; Week 3 asked for a full managed Kubernetes cluster, a working node pool, and a real application on top of it with persistent, block-volume-backed storage and public exposure through a cloud load balancer, all built twice, independently, across two separate build-and-teardown cycles.

I cover three things: the OKE-specific concepts behind my design (managed control plane, VCN-native pod networking, worker image selection, the Container Storage Interface and Cloud Controller Manager integrations that let Kubernetes provision real OCI infrastructure on its own), the Terraform mechanics I used to build the cluster from two reusable modules rather than one monolithic configuration, and the Kubernetes manifests and live debugging that took the deployment from a set of resources that merely existed to a request path that actually worked end to end.

Four findings stand out:

- **The cluster and node pool came up clean on two independent builds.** I tore the entire environment down at the end of the first working session to avoid overnight cost, and rebuilt it from the same code the next day with a single `terraform apply`, byte-for-byte equivalent to the original.
- **Worker image selection needed real, verified logic, not a static reference.** A generic image catalog lookup returned nothing usable, and an early version of the fix silently selected a GPU-variant image; the working version filters and sorts the purpose-built `node_pool_option` catalog and was verified independently against a captured candidate list before being trusted against live infrastructure.
- **The application only worked end to end after two live, non-trivial Kubernetes-layer bugs were diagnosed and fixed,** not merely configured: a container-runtime image-reference rejection, and a rollout deadlock caused by a single-replica pod holding a volume that only ever attaches to one node at a time.

- **Cloud resources that Kubernetes provisions on its own are invisible to Terraform**, and the environment's second, final teardown had to delete the Service and PersistentVolumeClaim through `kubectl` first, before `terraform destroy` ever ran, to avoid orphaning a real OCI Load Balancer and block volume that Terraform has no record of.

I fully implemented the intended deployment: a managed OKE cluster and two-node pool provisioned by Terraform, an nginx-based application with a proof-of-life initContainer running on top of it, a 50Gi OCI block volume attached through a PersistentVolumeClaim, and public exposure through a real OCI Load Balancer, confirmed working from an ordinary browser against the load balancer's public IP. Three limitations qualify that conclusion:

- the deployment runs a single application replica, a direct consequence of the storage's Read-WriteOnce access mode, not a scaling decision;
- I stored Terraform state only on my own local disk, with no locking and no shared source of truth; and
- I have not exercised the deployment under concurrent multi-operator use, sustained traffic, or a node or availability-domain failure.

Before I reuse this pattern for a team or a production workload, I would make three changes first: **split the storage-owning pod from a separately scalable stateless tier**, **move the state backend to OCI Object Storage**, and **add an Ingress controller with TLS termination** in front of the bare HTTP load balancer.

Contents

List of Abbreviations	vi
List of Figures	viii
List of Tables	ix
1 Introduction and Scope	1
1.1 Background and Motivation	1
1.2 Objective	2
1.3 Scope and Boundaries	2
1.4 Key Stakeholders	3
1.5 Main Risks and Uncertainties	3
1.6 How This Documentation Is Organized	4
2 Architecture Concepts and Trade-offs	5
2.1 What a Managed Control Plane Changes	5
2.2 Pod Networking: VCN-Native Versus Overlay	6
2.3 Storage: Persistent Volumes, Claims, and Access Modes	7
2.4 Cloud Integrations: the CSI Driver and the Cloud Controller Manager	8
2.5 Container Runtime Behavior Is Not Uniform Across Node Images	8
2.6 Evidence and Verification Basis	9
2.7 Worker Image Discovery: Two Approaches Compared	9
3 Terraform and Kubernetes Implementation	11
3.1 Two Reusable Modules Instead of One Configuration	11
3.2 Variables and tfvars	12
3.3 The Subnet Module: Dynamic Blocks and a Lifecycle Precondition	13
3.4 The OKE Module: Mutually Exclusive Pod Networking Blocks	16
3.5 Composing the Root Configuration	19
3.6 Worker Image Selection: Three Iterations	21
3.7 Kubernetes Manifests	23

3.7.1	Namespace, StorageClass, and PersistentVolumeClaim	23
3.7.2	Deployment: Rollout Strategy and Image References	25
3.7.3	Service	27
3.8	Environment Teardown	28
4	Deployment Results and Verification	29
4.1	The Deployed Network and Cluster Topology	29
4.2	Two Independent Build Cycles, Verified	32
4.3	Reaching the Cluster: An IAM Gap in the OCI CLI's Setup Flow	33
4.4	Two Live Application-Layer Bugs, Diagnosed and Fixed	35
4.4.1	CRI-O Rejects Unqualified Image References	35
4.4.2	A ReadWriteOnce Volume Deadlocks the Default Rollout Strategy . .	37
4.5	The Application Verified End to End	40
4.6	Environment Teardown, Executed Twice	42
4.7	Risk Analysis	45
4.8	Key Unknowns	45
5	Conclusions	47
5.1	Main Conclusion	47
5.2	Strongest Supporting Evidence	47
5.3	Biggest Uncertainty	48
5.4	Strategic Implications and Recommendation	48
5.5	Confidence Level and Final Judgement	48
6	Recommendations and Next Steps	50
6.1	Further Testing Needed	50
6.2	Data to Validate	50
6.3	Recommended Actions	50
6.4	Monitoring Indicators	51
6.5	Open Questions	52
Appendix		53
.1	Glossary	54
.2	Command Reference	55
References		56

List of Abbreviations

OCI	Oracle Cloud Infrastructure
OKE	Oracle Kubernetes Engine (Container Engine for Kubernetes)
VCN	Virtual Cloud Network
CIDR	Classless Inter-Domain Routing
IGW	Internet Gateway
NAT	Network Address Translation
CNI	Container Network Interface
CSI	Container Storage Interface
CCM	Cloud Controller Manager
CRI	Container Runtime Interface
PV	PersistentVolume
PVC	PersistentVolumeClaim
RWO	ReadWriteOnce
RWX	ReadWriteMany
AD	Availability Domain
LB	Load Balancer
IaC	Infrastructure as Code
HCL	HashiCorp Configuration Language
OCID	Oracle Cloud Identifier
VPU	Volume Performance Unit
SSH	Secure Shell

HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
API	Application Programming Interface
CLI	Command-Line Interface
JSON	JavaScript Object Notation
YAML	YAML Ain't Markup Language
DNS	Domain Name System
URL	Uniform Resource Locator
VM	Virtual Machine
IP	Internet Protocol
IAM	Identity and Access Management

List of Figures

3.1	terraform plan: node pool resource about to be created	19
3.2	terraform plan: worker image selection after the fix	23
4.1	Full Week 3 environment topology	30
4.2	terraform apply: cluster creation complete, node pool starting	32
4.3	terraform apply: final outputs	33
4.4	OCI CLI: browser-based login completed	34
4.5	OCI CLI: uploadapikey failure, HTTP 404	34
4.6	kubectl describe pod: ImageInspectError on the nginx container	36
4.7	kubectl describe pod: initContainer confirms the CRI-O runtime	37
4.8	kubectl get pods: two pods stuck, neither progressing	38
4.9	kubectl get pods -w: the full rollout-deadlock sequence	39
4.10	Browser request mid-incident: stale pod content served	40
4.11	kubectl get pvc: PersistentVolumeClaim Bound	41
4.12	kubectl get all: final healthy state	42
4.13	kubectl delete: StorageClass and namespace removed	43
4.14	terraform destroy: teardown complete	44

List of Tables

1.1	Stakeholders in the Week 3 build and this documentation.	3
2.1	Division of responsibility between OCI-managed and Terraform-provisioned components in an OKE deployment.	6
2.2	VCN-native pod networking versus the Flannel overlay alternative.	7
2.3	The three layered Kubernetes storage abstractions used in this build.	7
2.4	Evidence types used to support claims in this documentation.	9
2.5	Comparison of the two approaches to discovering a valid OKE worker image in Terraform.	10
3.1	The two Terraform modules this build is composed from.	12
3.2	Selected variables declared in <code>weekThree/variables.tf</code> , beyond the five provider-auth variables.	13
3.3	Three iterations of the worker-image selection logic.	21
4.1	CIDR allocation for the deployed VCN.	29
4.2	Endpoint subnet ingress rules. Egress: Kubernetes API to kubelet (TCP 10250) and to OCI services (TCP 443).	31
4.3	Workers subnet ingress rules. Egress: all protocols to 0.0.0.0/0 (image pulls, OCI API calls).	31
4.4	Pods subnet ingress rules. Egress: all protocols to 0.0.0.0/0.	31
4.5	LB subnet ingress rules. Egress: all protocols to the workers subnet (NodePort health checks and traffic).	31
4.6	Risks identified in the current configuration.	45
6.1	Indicators worth monitoring if this environment moves beyond a single-operator lab.	51
2	Terms used throughout this documentation.	54
3	Terraform and Kubernetes commands referenced in this documentation, all run against <code>weekThree/</code> during my work on this lab.	55

Chapter 1

Introduction and Scope

In Week 2 of this internship, I built a defense-in-depth network on Oracle Cloud Infrastructure (OCI): public and private subnets, a load balancer fronting a private application tier, and a managed Bastion for administrative access, all provisioned with Terraform. Week 3's assignment, *Kubernetes on OCI*, asked for something structurally different rather than merely bigger: a managed Kubernetes cluster, Oracle Kubernetes Engine (OKE), provisioned with Terraform, carrying a real application deployed with `kubectl`, with persistent block-volume-backed storage and public exposure through a cloud load balancer. This documentation is my combined deliverable for that assignment. I cover the OKE and Kubernetes concepts behind my build, the Terraform mechanics I used to provision the cluster from reusable modules, and the Kubernetes manifests and live debugging that took the deployment from resources that merely existed to a request path that actually worked, as one cross-referenced write-up rather than several separate ones.

1.1 Background and Motivation

A managed Kubernetes service is a **different layer of abstraction** from the VCNs, subnets, and compute instances I worked with in Weeks 1 and 2: OCI still owns the physical infrastructure, but Kubernetes itself, not Terraform, becomes the thing scheduling workloads, managing storage attachment, and provisioning the load balancer in front of the application. That split matters for how I structured this build. Terraform's job stops once the cluster and its node pool exist and are healthy; everything that runs *inside* the cluster, the namespace, the storage class, the persistent volume claim, the deployment, and the service, is defined separately as plain Kubernetes manifests and applied with `kubectl`. I kept those two layers in genuinely separate files with separate `apply` commands specifically so it stays easy to reason about what changes when: a `terraform apply` never touches a running pod, and a `kubectl apply` never touches the VCN or the node pool. The architecture chapter explains *what* each layer owns

and why, and the approach and findings chapters explain *how* I built and verified both of them together.

1.2 Objective

The primary question I set out to answer is: *does the OKE cluster I provisioned with Terraform actually run a real, verifiable application end to end, with genuine persistent storage and genuine public exposure, and is the Terraform configuration that provisions it reusable enough to rebuild the whole environment from scratch without manual intervention?* Two secondary questions follow from it:

- which worker node image selection strategy is actually reliable against OCI's rotating catalog of OKE-qualified images, given that a naive approach failed twice before I found one that worked; and
- which Kubernetes rollout strategy is safe for a pod bound to storage that can only ever attach to one node at a time.

1.3 Scope and Boundaries

I scoped this build to a **single OCI region**, `me-jeddah-1`, and a **single compartment**, `intern-02-ali-youssef-cmp`, using OCI's **VCN-native pod networking** mode rather than the Flannel overlay alternative, so pods receive real, routable addresses from a dedicated subnet rather than addresses that exist only inside an overlay invisible to the VCN. The application itself is deliberately simple: a single-replica nginx deployment serving a static page rendered by an initContainer, backed by one PersistentVolumeClaim. That is a *deliberate simplification* to keep the Kubernetes layer legible while the OKE and Terraform layers carry most of the complexity, not a shortcut I took by accident; Section 5.4 and Chapter 6 both address how I would extend it. I followed the `tf-lab-*--AliYousef` naming convention I established in Week 1 for every Terraform resource, and used the fixed application namespace `week3-demo` for every Kubernetes object. I built and fully tore down this environment **twice**: once on the day I finished the Terraform layer, torn down immediately afterward to avoid overnight cost before the application layer was deployed, and once more, completely independently, the following day, when I rebuilt the cluster from the same code and carried the deployment all the way through to a verified, publicly reachable application. I did not cover multi-region or multi-availability-domain high availability, horizontal scaling of the application tier, load testing under real traffic, or a production compliance review in this build. I name those explicitly as follow-up work in Chapter 6.

1.4 Key Stakeholders

Table 1.1: Stakeholders in the Week 3 build and this documentation.

Stakeholder	Interest or Role	Relevance
Operator (Ali Yousef)	Designs, deploys, and debugs the environment via Terraform and <code>kubectl</code>	Author of both the configuration and this documentation; the primary reader of the approach and findings chapters
Ejada Cloud Build Track	Reviews the assignment deliverable against the stated learning objectives	Target audience for this documentation; cares most about correctness of the OKE architecture and clarity of the Kubernetes debugging narrative
Application end users	Send HTTP requests to the load balancer's public IP	Affected by the correctness of the Service, Deployment, and readiness probe configuration described in Chapter 4
Future maintainers	Would extend this configuration, for example by splitting the application into a scalable tier or adding an Ingress controller	Rely on the rollout-strategy and storage-access-mode reasoning in Chapter 3 to change the workload safely

1.5 Main Risks and Uncertainties

Three issues are worth flagging before the detailed walkthrough in Chapter 4:

- The application runs at **a single replica**, a direct consequence of the PersistentVolumeClaim's ReadWriteOnce access mode, not a scaling decision I would defend for a workload that genuinely needed horizontal scale.
- I stored Terraform state only on my own local disk, with **no locking and no shared source of truth**, the same limitation I carried over unresolved from Week 2.
- I verified the environment functionally correct end to end, including a real browser request against the load balancer's public IP that returned the application's live content, but I have *not* exercised it under concurrent access, sustained load, or a failure of the single availability domain it lives in.

1.6 How This Documentation Is Organized

In Chapter 2, I set out the conceptual context: how OKE’s managed control plane changes the division of responsibility between Terraform and Kubernetes, the VCN-native pod networking model, the PersistentVolume and StorageClass abstractions, and the evidentiary basis for every claim in this documentation. In Chapter 3, I explain the Terraform mechanics I used to provision the cluster: the two modules I built it from, the worker-image selection logic and the two failed iterations that preceded the working one, and the Kubernetes manifests I applied on top of it. Chapter 4 walks through the environment as I actually deployed and verified it across both build cycles, including console and terminal evidence and the two real Kubernetes-layer bugs I diagnosed and fixed live. Chapter 5 turns the evidence into my final judgement, and Chapter 6 lists the concrete next actions and open questions I would tackle next. The appendix collects a glossary of terms and a command reference.

Chapter 2

Architecture Concepts and Trade-offs

In this chapter, I set out the conceptual background I relied on before building anything: what a managed Kubernetes control plane actually manages on my behalf, the pod networking and storage abstractions that decide how Kubernetes and OCI hand off responsibility to each other, the trade-offs between the practical options for discovering a valid worker node image, and the basis on which I ask you to trust every claim in this documentation.

2.1 What a Managed Control Plane Changes

Running Kubernetes yourself means operating the control plane components, the API server, etcd, the scheduler, and the controller manager, as a set of highly available services you patch, back up, and upgrade. OKE removes that layer entirely: OCI operates the control plane, and what I provision with Terraform is narrower than it sounds, summarized in Table 2.1.

Table 2.1: Division of responsibility between OCI-managed and Terraform-provisioned components in an OKE deployment.

Layer	Who operates it	What I actually provision
Control plane (API server, etcd, scheduler)	OCI, fully managed	Nothing directly; I reference the cluster by the OCID Terraform returns after creation
Worker nodes	OCI provisions the compute instances; I choose shape, count, and image	<code>oci_containerengine_node_pool</code> , plus the image-selection logic in Section 3.3
Pod networking	The CNI plugin, chosen at cluster-create time	A <code>dynamic</code> block selecting VCN-native or Flannel mode in Section 2.2
Cloud integrations (load balancers, block volumes)	OKE's built-in Cloud Controller Manager and CSI driver, watching Kubernetes objects	Nothing in Terraform; these are created automatically once I apply a <code>LoadBalancer Service</code> or a <code>PersistentVolumeClaim</code> , covered in Section 2.4

The row that matters most for how I structured this whole build is the last one. Once the cluster exists, **Kubernetes itself, not Terraform, becomes the thing that reaches back out to the OCI API** to provision a load balancer or a block volume, in direct response to a plain Kubernetes manifest I apply with `kubectl`. Terraform never sees those resources and has no record of them in its state file, a fact with real operational consequences I return to directly in Section 2.4 and again in Section 3.8.

2.2 Pod Networking: VCN-Native Versus Overlay

OKE offers two mutually exclusive pod networking modes, compared in Table 2.2.

Table 2.2: VCN-native pod networking versus the Flannel overlay alternative.

	VCN-native (OCI_VCN_IP_NATIVE)	Flannel overlay (FLANNEL_OVERLAY)
Pod IP address	A real, routable address assigned directly from a dedicated VCN subnet	An address that exists only inside an overlay network, invisible to the VCN itself
Address space needed	A large, dedicated pod subnet, since every pod consumes a real VCN address	Small; overlay addresses do not consume VCN address space
Visibility to VCN-level tooling (security lists, flow logs, NSGs)	Full: a pod's traffic is ordinary VCN traffic	Limited: VCN-level tooling only sees the encapsulated overlay traffic, not individual pod flows
Typical use case	Environments that want pods to be first-class, directly addressable VCN citizens	Environments prioritizing address-space conservation over per-pod VCN visibility

I chose VCN-native pod networking for this build specifically because it keeps pod traffic visible to the same VCN-level security and logging tooling I already relied on in Week 2, at the cost of dedicating a genuinely large subnet, `10.0.32.0/19`, to pods alone. Both the cluster resource and the node pool resource need matching, mutually exclusive `dynamic` blocks to express this choice, which I cover as a Terraform mechanism in Section 3.4.

2.3 Storage: PersistentVolumes, Claims, and Access Modes

Kubernetes storage is built from three layered abstractions, summarized in Table 2.3, and understanding the layering matters directly for one of the two real bugs I hit in this lab (Section 4.4.2).

Table 2.3: The three layered Kubernetes storage abstractions used in this build.

Abstraction	Role
StorageClass	A template telling Kubernetes which driver to use and with what parameters when it needs to dynamically provision storage; not itself a piece of storage.
PersistentVolumeClaim (PVC)	An namespaced <i>request</i> for storage matching certain criteria (size, access mode, StorageClass); what an application's pod spec actually references.
PersistentVolume (PV)	The real, provisioned storage the PVC is bound to; created automatically by the CSI driver in response to the PVC, backed here by a real OCI block volume.

The access mode is the detail that matters most here. OCI block volumes are fundamen-

tally **ReadWriteOnce** devices: the underlying volume can be attached read-write to exactly one node at a time, a property of the storage technology itself, not a policy choice Kubernetes imposes. The alternative access modes, **ReadWriteMany** and **ReadOnlyMany**, require a genuinely different, typically network-file-system-backed storage technology, such as OCI File Storage Service, not block volumes. Every consequence of that constraint, including a rollout deadlock I hit live, traces back to this one fact.

2.4 Cloud Integrations: the CSI Driver and the Cloud Controller Manager

Two components, pre-installed on every OKE cluster, are what let a plain Kubernetes manifest reach back out and provision real OCI infrastructure without any Terraform involvement.

- **The Block Volume CSI driver** (Container Storage Interface) watches for `PersistentVolumeClaims` referencing its provisioner, `blockvolume.csi.oraclecloud.com`, and calls the OCI Block Volume API directly to create and attach a real volume in response.
- **The Cloud Controller Manager (CCM)** watches for Services of type: `LoadBalancer` and calls the OCI Load Balancing API directly to provision a real, internet-facing load balancer in response, reading OCI-specific configuration from `service.beta.kubernetes.io/oci-load-balancer-*` annotations on the Service object.

Both mechanisms are the general Kubernetes cloud-provider-integration pattern: the cluster reaches out and manages real cloud infrastructure on the application's behalf, entirely outside of Terraform's state. I return to the direct operational consequence of this, specifically for environment teardown, in Section 3.8.

2.5 Container Runtime Behavior Is Not Uniform Across Node Images

One operational constraint shaped the application layer beyond the Kubernetes abstractions themselves: **the container runtime on OKE's worker nodes is CRI-O, and this particular node image enforces strict short-name resolution for image references.** A short image reference such as `nginx:1.27-alpine`, with no registry stated explicitly, is ambiguous, since it could in principle resolve against Docker Hub or any other configured registry, and CRI-O in enforcing mode refuses to guess rather than risk resolving to the wrong registry. This is a real, documented container-security posture, not an arbitrary restriction, and it directly caused one of the two live bugs I diagnose in Section 4.4.1. I revisit this constraint directly in the deployed configuration I examine in Chapter 4.

2.6 Evidence and Verification Basis

I did not build this documentation from external literature. It covers one specific environment I deployed myself, so every claim needs a different kind of grounding: reproducible evidence from the environment itself rather than a citation. Table 2.4 summarizes the three kinds of evidence I use throughout this documentation, each referenced by figure or table number at the point it supports a claim.

Table 2.4: Evidence types used to support claims in this documentation.

Evidence type	What it confirms	Basis for trust
Terraform apply and plan output, captured directly from my terminal	Exact attribute values OCI assigned to each resource, and the real timing of long-running operations	Written directly by the running <code>terraform</code> process at the time of the build, not hand-typed
<code>kubectl</code> output (<code>get</code> , <code>describe</code> , <code>get -w</code>)	The real, live state of every Kubernetes object, including error conditions as they actually occurred	Executed directly against the live cluster during this lab, including mid-incident while diagnosing both bugs in Chapter 4
A live browser request against the load balancer’s public IP	That the full request path works end to end, not just that the resources exist	Captured directly from the deployed environment; the page itself renders the serving pod’s own hostname and a live timestamp

2.7 Worker Image Discovery: Two Approaches Compared

Every OKE node pool needs a worker image compatible with both the node shape and the cluster’s Kubernetes minor version, and OCI does not publish a single, stable image OCID to reference: the set of valid images rotates as OCI ships new builds. Two Terraform-native approaches to discovering one are compared in Table 2.5 before I explain, in Chapter 3, which one I used, why the first attempt with it still failed twice, and how I finally made it reliable.

Table 2.5: Comparison of the two approaches to discovering a valid OKE worker image in Terraform.

Approach	How it works	Reliability
Generic image catalog (oci_core_images)	Query OCI's general-purpose compute image catalog, filtered by a display-name pattern	Unreliable for OKE specifically: this catalog is not guaranteed to completely or consistently represent OKE-qualified images, and returned zero results when I filtered it for one
Purpose-built node pool option catalog (oci_containerengine_node_pools_options)	Query the catalog OKE itself uses to validate worker images for a given cluster, Kubernetes version, and shape	Reliable in principle, but the raw candidate list still needed real filtering logic to exclude wrong architectures and GPU variants, and to prefer a version-tagged image over an untagged rolling one

The pattern to notice: the purpose-built catalog is the right data source, but it is not sufficient by itself. Even querying the correct catalog, I still needed real filtering and sorting logic on top of it, and got that logic wrong twice before verifying a third version independently. That progression is the subject of Section 3.6.

Chapter 3

Terraform and Kubernetes Implementation

In Week 2, I used Terraform at the level of a single flat configuration with typed variables, locals, and `for_each`. Week 3's assignment asked for more structure: a configuration that is

- **modular**, built from reusable pieces rather than one long file,
- **correct against a rotating external catalog**, since OKE's set of valid worker images changes as OCI ships new builds, and
- **layered correctly with Kubernetes**, so the boundary between what Terraform owns and what `kubectl` owns stays clean.

In this chapter, I explain each mechanism I used to meet those requirements; every code listing below is copied verbatim from my files in `weekThree/`, not simplified or rewritten for this documentation.

3.1 Two Reusable Modules Instead of One Configuration

I split this build into two Terraform modules, summarized in Table 3.1, called from a root configuration that composes them.

Table 3.1: The two Terraform modules this build is composed from.

Module	What it creates	Called
<code>./modules/subnet</code>	A subnet, its route table, its security list, and an optional VCN flow log	Four times, once per subnet tier (endpoint, workers, pods, load balancer)
<code>./modules/oke</code>	The OKE cluster and its managed node pool	Once

The `subnet` module is deliberately generic: it has no idea it is being used to build an OKE environment specifically. It accepts route rules, ingress rules, and egress rules as typed variables and renders them with `dynamic` blocks, so the same module definition produces four subnets with entirely different rule sets from four different call sites in the root configuration, rather than four near-identical, hand-written resource blocks. The `oke` module is narrower and purpose-built: it wraps exactly the two resources an OKE deployment needs, the cluster and its node pool, and exposes the specific inputs, pod networking mode, worker image ID, node shape, that the root configuration needs to control per environment.

3.2 Variables and tfvars

As in Week 2, every resource file references a named, typed input rather than a literal value, and I supply real values from `terraform.tfvars`, listed in `.gitignore` and never committed, since it holds the actual tenancy OCID, user OCID, API key fingerprint, and private key path, each marked `sensitive = true` in `variables.tf`. Table 3.2 lists the variables specific to this week's OKE and application layer, beyond the five provider-auth variables carried over unchanged from Week 1 and Week 2.

Table 3.2: Selected variables declared in `weekThree/variables.tf`, beyond the five provider-auth variables.

Variable	Type	Purpose
<code>vcn_cidr_block</code>	string	VCN CIDR, default <code>10.0.0.0/16</code>
<code>endpoint_subnet_cidr</code>	string	Kubernetes API endpoint subnet, default <code>10.0.0.0/28</code>
<code>workers_subnet_cidr</code>	string	Worker node subnet, default <code>10.0.16.0/20</code>
<code>pods_subnet_cidr</code>	string	Pod subnet, default <code>10.0.32.0/19</code> ; deliberately large since VCN-native networking assigns each pod a real VCN address
<code>lb_subnet_cidr</code>	string	Public load balancer subnet, default <code>10.0.8.0/24</code>
<code>is_endpoint_public</code>	bool	Whether the Kubernetes API endpoint gets a public IP, default <code>true</code> for direct <code>kubectl</code> access from a local machine
<code>node_pool_size</code>	number	Worker node count, default <code>2</code>
<code>node_shape</code>	string	Compute shape, default <code>VM.Standard.E4.Flex</code>
<code>node_ocpus, node_memory_in_gbs</code>	number	Flex-shape sizing, default <code>2 OCPUs / 16GB</code>
<code>max_pods_per_node</code>	number	Pod density cap under VCN-native networking, default <code>31</code>
<code>app_block_volume_size_in_gbs</code>	number	Size of the application's <code>PersistentVolumeClaim</code> , default <code>50</code> ; kept in sync by hand with <code>k8s/pvc.yaml</code>

3.3 The Subnet Module: Dynamic Blocks and a Lifecycle Precondition

The subnet module's route rules, ingress rules, and egress rules are each typed as a list of objects, with `optional()` fields for anything not every rule needs, so Terraform validates the shape of every rule at `terraform validate` time rather than surfacing a confusing error at apply time.

Listing 3.1: `modules/subnet/variables.tf` – the ingress rule type, illustrating the pattern used for all three rule variables

```

1 variable "ingress_rules" {
2   type = list(object({
3     source      = string
4     source_type = optional(string, "CIDR_BLOCK")
5     protocol    = string
6     description = optional(string, "")
7     stateless   = optional(bool, false)
  })
}
```

```

8     tcp_options = optional(object({
9         min = number
10        max = number
11    }))
12    udp_options = optional(object({
13        min = number
14        max = number
15    }))
16    icmp_options = optional(object({
17        type = number
18        code = optional(number)
19    }))
20    }))
21    default = []
22 }

```

Inside the module, each rule variable drives a `dynamic` block, and each rule’s optional port-range fields drive a *nested* dynamic block one level deeper, using the idiom of feeding a single-or-empty list, rather than a boolean, to `for_each` to express “render this nested block only if this optional field is actually set.”

Listing 3.2: `modules/subnet/main.tf` – the ingress security list, with a nested dynamic block for each optional port-range type

```

1 resource "oci_core_security_list" "this" {
2     compartment_id = var.compartment_id
3     vcn_id         = var.vcn_id
4     display_name   = var.security_list_display_name
5     freeform_tags  = var.freeform_tags
6
7     dynamic "ingress_security_rules" {
8         for_each = var.ingress_rules
9         content {
10             source          = ingress_security_rules.value.source
11             source_type     = ingress_security_rules.value.source_type
12             protocol        = ingress_security_rules.value.protocol
13             description     = ingress_security_rules.value.description
14             stateless       = ingress_security_rules.value.stateless
15
16             dynamic "tcp_options" {
17                 for_each = ingress_security_rules.value.tcp_options != null ? [
18                     ingress_security_rules.value.tcp_options] : []
19                 content {
20                     min = tcp_options.value.min
21                     max = tcp_options.value.max
22                 }
23             }
24         }
25     }
26 }

```



```

24     dynamic "udp_options" {
25         for_each = ingress_security_rules.value.udp_options != null ? [
26             ingress_security_rules.value.udp_options] : []
27         content {
28             min = udp_options.value.min
29             max = udp_options.value.max
30         }
31     }
32 }
33 % egress_security_rules mirrors this block exactly, substituting
34   destination for source.

```

`tcp_options.value != null ? [tcp_options.value] : []` is the idiom worth naming explicitly: a dynamic block's `for_each` needs a list or map, not a boolean, so “render this block only if this value is set” is expressed as “iterate a list that is either empty or has exactly one element.” The same rule object works whether it specifies a TCP port range, a UDP port range, both, or neither, with no separate resource block needed for each combination.

The module's optional flow-log resource also demonstrates a validation mechanism Terraform does not otherwise provide directly: enforcing that one variable is set whenever *another* variable has a particular value.

Listing 3.3: `modules/subnet/main.tf` – the optional flow log, gated by `count` and validated by a lifecycle precondition

```

1 resource "oci_logging_log" "flow_log" {
2     count = var.enable_logs ? 1 : 0
3
4     display_name = "${var.subnet_display_name}-flow-log"
5     log_group_id = var.log_group_id
6     log_type      = "SERVICE"
7
8     configuration {
9         compartment_id = var.compartment_id
10        source {
11            category      = "all"
12            resource      = oci_core_subnet.this.id
13            service        = "flowlogs"
14            source_type    = "OCISERVICE"
15        }
16    }
17
18    retention_duration = var.log_retention_duration
19    is_enabled         = true
20    freeform_tags      = var.freeform_tags
21

```

```

22 lifecycle {
23   precondition {
24     condition      = var.log_group_id != null
25     error_message = "log_group_id must be set when enable_logs = true."
26   }
27 }
28 }

```

I first tried to express this constraint as a validation block directly on the `log_group_id` variable itself, which failed to plan: a variable's own validation block can only reference that same variable's value, not a second variable like `enable_logs`. `lifecycle { precondition { } }`, attached to the resource that actually needs the invariant to hold, is the correct mechanism, since a precondition's `condition` expression runs in the context of the full resource configuration and can freely reference as many input variables as it needs. This is the first of two real configuration mistakes I made and fixed during this build; the second, which took two more iterations to get right, is the worker image selection logic in Section 3.6.

3.4 The OKE Module: Mutually Exclusive Pod Networking Blocks

The choice between VCN-native and Flannel pod networking (Section 2.2) has to be expressed identically, and consistently, in two separate resources: the cluster itself, and the node pool. Both use the same `dynamic-block` idiom, this time choosing between two *mutually exclusive* dynamic blocks rather than rendering an optional one.

Listing 3.4: `modules/oke/main.tf` – the cluster's pod network options: exactly one of these two blocks ever renders

```

1 resource "oci_containerengine_cluster" "this" {
2   compartment_id = var.compartment_id
3   vcn_id         = var.vcn_id
4   name          = var.cluster_name
5   kubernetes_version = var.kubernetes_version
6   type          = var.cluster_type
7
8   endpoint_config {
9     is_public_ip_enabled = var.is_endpoint_public
10    subnet_id             = var.endpoint_subnet_id
11    nsg_ids               = var.endpoint_nsg_ids
12  }
13
14  dynamic "cluster_pod_network_options" {
15    for_each = var.pod_network_type == "VCN_NATIVE" ? [1] : []
16    content {

```

```

17     cni_type = "OCI_VCN_IP_NATIVE"
18   }
19 }
20
21 dynamic "cluster_pod_network_options" {
22   for_each = var.pod_network_type == "FLANNEL_OVERLAY" ? [1] : []
23   content {
24     cni_type = "FLANNEL_OVERLAY"
25   }
26 }
27
28 options {
29   service_lb_subnet_ids = var.service_lb_subnet_ids
30   admission_controller_options {
31     is_pod_security_policy_enabled = var.enable_pod_security_policy
32   }
33   kubernetes_network_config {
34     pods_cidr      = var.pod_network_type == "FLANNEL_OVERLAY" ? var.
35       pods_cidr : null
36     services_cidr = var.services_cidr
37   }
38 }
39 freeform_tags = var.freeform_tags
40 }

```

The node pool needs the equivalent choice made a second time, inside its own `node_config_details` block, because the CNI mode affects how the node pool itself requests pod IP addresses, not just how the cluster's control plane is configured.

Listing 3.5: `modules/oke/main.tf` – the node pool's matching pod network option, plus placement and image source

```

1 resource "oci_containerengine_node_pool" "this" {
2   cluster_id      = oci_containerengine_cluster.this.id
3   compartment_id  = var.compartment_id
4   name            = var.node_pool_name
5   kubernetes_version = var.kubernetes_version
6   node_shape      = var.node_shape
7
8   dynamic "node_shape_config" {
9     for_each = var.node_shape_config != null ? [var.node_shape_config] :
10       []
11     content {
12       ocpus      = node_shape_config.value.ocpus
13       memory_in_gbs = node_shape_config.value.memory_in_gbs
14     }
15   }
16 }

```

```
15
16 node_source_details {
17     source_type      = "IMAGE"
18     image_id         = var.node_image_id
19     boot_volume_size_in_gbs = var.boot_volume_size_in_gbs
20 }
21
22 node_config_details {
23     size = var.node_pool_size
24
25     dynamic "placement_configs" {
26         for_each = var.node_placement_configs
27         content {
28             availability_domain = placement_configs.value.availability_domain
29             subnet_id          = placement_configs.value.subnet_id
30         }
31     }
32
33     dynamic "node_pool_pod_network_option_details" {
34         for_each = var.pod_network_type == "VCN_NATIVE" ? [1] : []
35         content {
36             cni_type      = "OCI_VCN_IP_NATIVE"
37             pod_subnet_ids = var.pod_subnet_ids
38             pod_nsg_ids    = var.pod_nsg_ids
39             max_pods_per_node = var.max_pods_per_node
40         }
41     }
42
43     dynamic "node_pool_pod_network_option_details" {
44         for_each = var.pod_network_type == "FLANNEL_OVERLAY" ? [1] : []
45         content {
46             cni_type = "FLANNEL_OVERLAY"
47         }
48     }
49 }
50
51 # Omitting ssh_public_key entirely, rather than passing an empty string
52 '
53 # is what actually disables SSH access on OKE-managed nodes.
54 ssh_public_key = var.ssh_public_key != "" ? var.ssh_public_key : null
55
56 initial_node_labels {
57     key    = "name"
58     value = var.node_pool_name
59 }
60
61 freeform_tags = var.freeform_tags
62 }
```

Figure 3.1 is the actual terraform plan diff for this resource against my live rebuild, showing `node_pool_pod_network_option_details` rendered with `cni.type = "OCI_VCN_IP_NATIVE"` and `max_pods_per_node = 31`, matching the configuration above exactly rather than a simplified illustration of it.

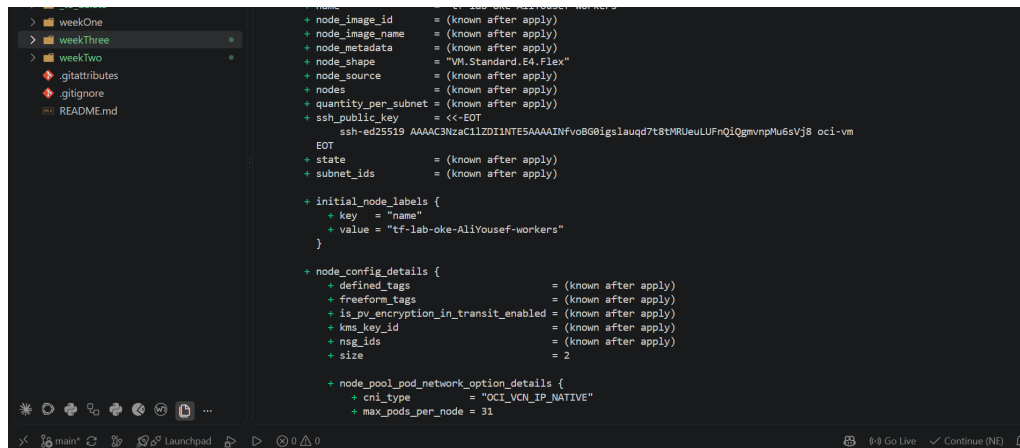


Figure 3.1: terraform plan, captured live: the node pool resource being created, showing `node_shape = "VM.Standard.E4.Flex"`, `size = 2`, and `node_pool_pod_network_option_details { cni.type = "OCI_VCN_IP_NATIVE", max_pods_per_node = 31 }` exactly as declared in `modules/oke/main.tf`.

3.5 Composing the Root Configuration

The root configuration calls the `subnet` module four times, once per tier, and the `oke` module once, wiring the two together. One call site is worth walking through in full, because it demonstrates a fourth conditional pattern beyond the three already covered: adding an entire *list element* to a module call's input conditionally, at the call site, rather than inside the module.

Listing 3.6: `subnets.tf` – the endpoint subnet's ingress rules, showing the `concat()` conditional-element pattern

```
1 module "endpoint_subnet" {
2   source = "../modules/subnet"
3
4   compartment_id      = local.my_compartment_id
5   vcn_id               = oci_core_vcn.this.id
6   subnet_display_name = "tf-lab-oke-endpoint-subnet-AliYousef"
7   subnet_cidr_block   = var.endpoint_subnet_cidr
8   is_public            = var.is_endpoint_public
9
10  # concat() with a conditionally-empty list is the pattern for "maybe
11  # this rule" when the module takes a plain list, not a dynamic block -
```

```

12  # the dynamic block rendering happens inside the module, not here.
13  ingress_rules = concat(
14    [
15      {
16        source      = var.workers_subnet_cidr
17        protocol    = "6"
18        description = "Worker nodes -> Kubernetes API"
19        tcp_options = { min = 6443, max = 6443 }
20      },
21      {
22        source      = var.workers_subnet_cidr
23        protocol    = "6"
24        description = "Worker nodes -> OKE control plane services"
25        tcp_options = { min = 12250, max = 12250 }
26      }
27    ],
28    var.is_endpoint_public ? [
29      {
30        source      = "0.0.0.0/0"
31        protocol    = "6"
32        description = "External kubectl access to the Kubernetes API"
33        tcp_options = { min = 6443, max = 6443 }
34      }
35    ] : []
36  )
37  # route_rules and egress_rules omitted here for brevity; full rule
   tables
38  # for every subnet, including this one, appear in Chapter~\ref{chap:
   results}.
39  }

```

This is a genuinely different mechanism from the `dynamic` block covered in Section 3.3, even though both exist to answer the same kind of question, “should this optional thing be included.” A `dynamic` block operates *inside* a resource or module, expanding a list into repeated nested blocks at that resource’s own level. `concat()` with a conditionally-empty list operates *at the call site*, before the module even runs, deciding what goes into the ordinary list variable the module receives. I needed the call-site version here specifically because `ingress_rules` is a plain `list(object(...))` input to the module, not a nested block the module itself renders dynamically; the module has no idea whether the third rule exists, it simply iterates whatever list it was handed.

The root configuration’s other three subnet calls, the workers, pods, and load balancer subnets, follow the same module-call shape with different, fixed rule sets, all reproduced in full alongside their design rationale in Chapter 4, Section 4.1.

3.6 Worker Image Selection: Three Iterations

Every OKE node pool needs a worker image compatible with both its node shape and its cluster's Kubernetes minor version, and Section 2.7 already established why the generic `oci_core_images` catalog was the wrong data source to query for one. Table 3.3 summarizes the three iterations the actual selection logic went through before I trusted it against live infrastructure.

Table 3.3: Three iterations of the worker-image selection logic.

Iter.	Approach	Result
1	Query <code>oci_core_images</code> , filtered by an Oracle Linux display-name pattern	Zero results: this catalog does not reliably represent OKE-qualified images
2	Query <code>oci_containerengine_node_pool_option</code> , sort candidate names descending, take the first	Selected a real, OKE-compatible image, but one from the Gen2-GPU variant family, incompatible with the plain CPU-only shape actually in use
3 (final)	Query the same purpose-built catalog, explicitly exclude <code>aarch64</code> and GPU name substrings, prefer a version-tagged image over an untagged rolling one, verified independently before use	Correct: selected <code>Oracle-Linux-8.10-2026.06.15-0-OKE-1.36.1-1505</code> , verified against a standalone reimplement of the same filter logic

The final version is a layered piece of `locals` logic:

Listing 3.7: `oke.tf` – the complete, final worker-image selection logic

```

1 data "oci_containerengine_node_pool_option" "this" {
2   node_pool_option_id = "all"
3   compartment_id      = local.my_compartment_id
4 }
5
6 locals {
7   # Oracle Linux 8 worker images, name -> OCID, general-purpose x86 only.
8   # "aarch64" excludes Ampere/ARM builds; "GPU" excludes Gen2-GPU and
9   # Gen2-AMD-GPU builds. Both caused "Node shape and image are not
10  # compatible" against VM.Standard.E4.Flex before being excluded here.
11  oke_worker_candidates = {
12    for s in data.oci_containerengine_node_pool_option.this.sources :
13    s.source_name => s.image_id
14    if can(regex("^Oracle-Linux-8", s.source_name))
15      && !can(regex("aarch64", s.source_name))
16      && !can(regex("GPU", s.source_name))
17  }
18

```

```

19  # Prefer an image explicitly built for this cluster's Kubernetes
    version -
20  # name contains "-OKE-<version>-" - since that is the exact combination
21  # OCI validates together. Fall back to the newest candidate overall if
    no
22  # version-tagged image exists.
23  oke_version_tagged_images = {
24    for name, id in local.oke_worker_candidates : name => id
25    if can(regex("-OKE-${trimprefix(local.kubernetes_version, "v")}-",
        name))
26  }
27
28  oke_worker_images = length(local.oke_version_tagged_images) > 0 ? local
    .oke_version_tagged_images : local.oke_worker_candidates
29
30  # Names sort lexically the same as chronologically (build date embedded
    ),
31  # so the last element is the newest.
32  node_image_id = local.oke_worker_images[
33    sort(keys(local.oke_worker_images)) [length(local.oke_worker_images) -
        1]
34  ]
35  }

```

Four things happen in sequence here. First, `oke_worker_candidates` filters every image the data source returns down to general-purpose x86 Oracle Linux 8 builds, using `can(regex(...))` rather than a bare `regex(...)` match specifically so that a name which does not match a pattern evaluates cleanly to `false` rather than raising an error, since `regex()` on its own is designed to extract a value and errors on no match. Second, `oke_version_tagged_images` filters that candidate set further, to only images whose name contains the cluster's actual Kubernetes version. Third, `oke_worker_images` is a single ternary deciding between the version-tagged set and the full candidate set as a fallback, using `length(...) > 0` as the standard Terraform idiom for “is this collection non-empty.” Fourth, `node_image_id` sorts whichever set won and takes the lexically last key, which is safe because OCI's image names embed a `YYYY.MM.DD` build date that sorts identically whether compared as a string or as a date.

Figure 3.2 is the actual `terraform plan` output from the moment iteration 3 replaced iteration 2 in my live configuration: the diagnostic output `debug_selected_node_image` shows the exact transition, from a `Gen2-GPU-variant` image the old logic had selected, to the correct, version-tagged image the new logic selects, against the same live candidate list.

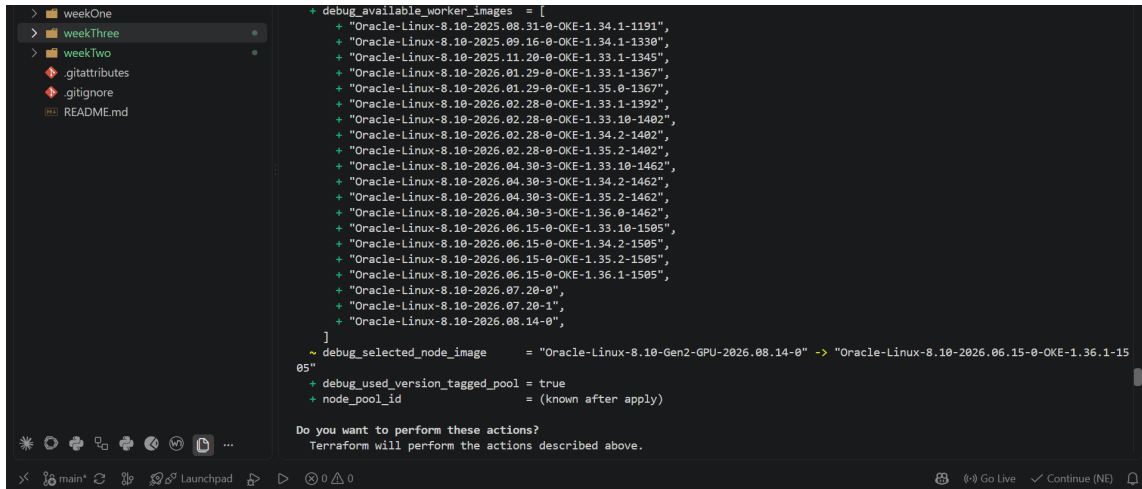


Figure 3.2: terraform plan, captured live: `debug_available_worker_images` lists every remaining x86, non-ARM Oracle Linux 8 candidate, and `debug_selected_node_image` shows the actual transition triggered by the iteration-3 fix, "Oracle-Linux-8.10-Gen2-GPU-2026.08.14-0" -> "Oracle-Linux-8.10-2026.06.15-0-OKE-1.36.1-1505".

I did not trust the final version on faith after the first two iterations had already failed. Before sending it back for a real terraform plan, I reimplemented the same filter and sort logic standalone in Python against the exact candidate list a prior failed plan had already printed through the diagnostic `debug_available_worker_images` output, and confirmed independently that it selected the version-tagged, non-ARM, non-GPU image, before trusting it against live infrastructure. That independent check is what Figure 3.2 then confirmed against the real environment.

3.7 Kubernetes Manifests

Terraform's responsibility ends once the cluster and node pool exist and are healthy. Everything that runs *inside* the cluster is defined separately, as five plain Kubernetes manifests in `k8s/`, applied in dependency order with `kubectl`: namespace, storage class, and PersistentVolumeClaim first, since the deployment's pod spec references the PVC by name, then the deployment and service.

3.7.1 Namespace, StorageClass, and PersistentVolumeClaim

Listing 3.8: `k8s/namespace.yaml`, in full

```
1 apiVersion: v1
2 kind: Namespace
3 metadata:
```

```
4 name: week3-demo
```

A dedicated namespace, rather than the cluster’s default, keeps this lab’s objects logically separated and makes teardown legible, covered directly in Section 3.8.

Listing 3.9: `k8s/storageclass.yaml`, in full

```
1 apiVersion: storage.k8s.io/v1
2 kind: StorageClass
3 metadata:
4   name: week3-oci-bv
5 provisioner: blockvolume.csi.oraclecloud.com
6 parameters:
7   vpusPerGB: "10"
8   attachment-type: paravirtualized
9 reclaimPolicy: Delete
10 volumeBindingMode: WaitForFirstConsumer
11 allowVolumeExpansion: true
```

I defined an explicit `StorageClass` rather than relying on OKE’s pre-installed `oci-bv` default, so the performance tier and reclaim behavior are visible and deliberate rather than inherited silently. `vpusPerGB: "10"` selects OCI’s “Balanced” block volume performance tier. `volumeBindingMode: WaitForFirstConsumer` delays actual volume provisioning until a pod referencing the PVC is scheduled to a specific node, which matters directly for availability-domain-scoped storage like an OCI block volume: provisioning `Immediately`, before Kubernetes knows which node will use the volume, risks creating it in the wrong availability domain. `reclaimPolicy: Delete` deletes the real underlying OCI block volume when its PVC is deleted, the correct choice for a disposable lab environment where the whole point of teardown is leaving nothing billable behind.

Listing 3.10: `k8s/pvc.yaml`, in full

```
1 apiVersion: v1
2 kind: PersistentVolumeClaim
3 metadata:
4   name: demo-app-data
5   namespace: week3-demo
6 spec:
7   accessModes:
8     - ReadWriteOnce
9   storageClassName: week3-oci-bv
10  resources:
11    requests:
12      storage: 50Gi
```

`accessModes: [ReadWriteOnce]` is not a policy choice on my part; it is what OCI block volumes support, and is the constraint behind both real bugs I fix in Chapter 4. `storage:`

50Gi is kept in sync by hand with `var.app_block_volume_size_in_gbs` in the Terraform root module, documented as such directly in the file, even though the two are not mechanically linked: this PVC's volume is created by the CSI driver, entirely outside anything Terraform provisions directly.

3.7.2 Deployment: Rollout Strategy and Image References

Listing 3.11: `k8s/deployment.yaml`, final working version, in full

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: demo-app
5   namespace: week3-demo
6   labels:
7     app: demo-app
8 spec:
9   replicas: 1
10  strategy:
11    type: Recreate
12  selector:
13    matchLabels:
14      app: demo-app
15  template:
16    metadata:
17      labels:
18        app: demo-app
19    spec:
20      initContainers:
21        - name: write-index
22          image: docker.io/library/busybox:1.36
23          command:
24            - sh
25            - -c
26            - |
27              cat <<EOF > /data/index.html
28              <html>
29                <head><title>Ejada Week 3 - OKE Demo</title></head>
30                <body style="font-family: sans-serif;">
31                  <h1>Ejada Internship - Week 3 OKE Lab</h1>
32                  <p>Served by pod: $(hostname)</p>
33                  <p>Node: $NODE_NAME</p>
34                  <p>Rendered at container start: $(date -u)</p>
35                  <p>This page is being read from a PersistentVolumeClaim
36                    backed
37                    by an OCI block volume, attached via the CSI driver
38                    .</p>
39                </body>
```

```
38         </html>
39         EOF
40     env:
41     - name: NODE_NAME
42       valueFrom:
43         fieldRef:
44           fieldPath: spec.nodeName
45     volumeMounts:
46     - name: data
47       mountPath: /data
48   containers:
49   - name: nginx
50     image: docker.io/library/nginx:1.27-alpine
51     ports:
52     - containerPort: 80
53     volumeMounts:
54     - name: data
55       mountPath: /usr/share/nginx/html
56       readOnly: true
57     resources:
58       requests:
59         cpu: 100m
60         memory: 128Mi
61       limits:
62         cpu: 250m
63         memory: 256Mi
64     readinessProbe:
65       httpGet:
66         path: /index.html
67         port: 80
68       initialDelaySeconds: 3
69       periodSeconds: 5
70     livenessProbe:
71       httpGet:
72         path: /index.html
73         port: 80
74       initialDelaySeconds: 10
75       periodSeconds: 10
76   volumes:
77   - name: data
78     persistentVolumeClaim:
79       claimName: demo-app-data
```

Two details in this manifest, `strategy: type: Recreate` and the fully-qualified `docker.io/library/...` image references, are not part of a first draft: both were added live, in response to real failures I diagnose in full in Chapter 4, Sections 4.4.1 and 4.4.2. I explain the reasoning behind each here, since both are genuine Terraform-adjacent implemen-

tation decisions, and diagnose the actual incidents that produced them in the findings chapter.

`replicas: 1` is a direct consequence of the `PersistentVolumeClaim`'s `ReadWriteOnce` access mode (Section 2.3): a second replica would either need to land on the same node as the first, defeating much of the point of a second replica, or fail outright if scheduled elsewhere. `strategy: type: Recreate` overrides Kubernetes' default `RollingUpdate` strategy, which brings a new pod up *before* tearing down the old one to achieve zero-downtime rollouts. That default is actively wrong for a single-replica pod bound to an RWO volume: the new pod's `initContainer` cannot start until the volume detaches from the node the old pod is still running on, and the old pod will not be torn down until the rollout considers the new pod healthy, a genuine circular deadlock. `Recreate` inverts the order, terminating the existing pod first, so the volume frees up before the replacement needs it, at the cost of a brief real downtime window on any future rollout, an accepted tradeoff for a demo application.

The image references are fully qualified with an explicit registry, `docker.io/library/busybox:1.36` and `docker.io/library/nginx:1.27-alpine`, rather than the shorter, unqualified form. Section 2.5 already covers why: OKE's worker nodes run CRI-O with short-name resolution set to enforcing, which rejects an unqualified reference outright rather than guessing a registry.

The `initContainer`'s Downward-API-sourced `NODE_NAME` environment variable and its here-document HTML generation exist purely to make the storage layer's correctness visible: the served page states which pod and node rendered it and when, which is what lets a browser request against the load balancer function as genuine end-to-end proof that the block volume is mounted and writable, not just that the pod shows `Running`.

3.7.3 Service

Listing 3.12: `k8s/service.yaml`, in full

```
1 apiVersion: v1
2 kind: Service
3 metadata:
4   name: demo-app-lb
5   namespace: week3-demo
6   annotations:
7     service.beta.kubernetes.io/oci-load-balancer-shape: "flexible"
8     service.beta.kubernetes.io/oci-load-balancer-shape-flex-min: "10"
9     service.beta.kubernetes.io/oci-load-balancer-shape-flex-max: "10"
10 spec:
11   type: LoadBalancer
12   selector:
13     app: demo-app
14   ports:
15     - port: 80
```

```
16     targetPort: 80
17     protocol: TCP
```

`type: LoadBalancer` is what triggers the Cloud Controller Manager (Section 2.4) to provision a real OCI Load Balancer automatically, entirely outside Terraform's awareness. The three `oci-load-balancer-*` annotations are OCI-specific configuration the Cloud Controller Manager reads when it does so: `shape: "flexible"` selects OCI's bandwidth-metered Flexible shape, and pinning both the minimum and maximum to 10 (Mbps) provisions the smallest, cheapest practical size, a deliberate cost-consciousness choice for a lab demo serving one small static page, consistent with the teardown discipline covered in Chapter 5.

3.8 Environment Teardown

I tore this environment down completely, twice, once at the end of the first working session before the application layer was even deployed, purely to avoid overnight cost, and once more at the very end, after every requirement was verified working. The second teardown needed a strict order that the first did not, because by then the cluster had real, Kubernetes-managed cloud resources attached to it that Terraform has no record of at all: the OCI Load Balancer created by the Cloud Controller Manager, and the OCI block volume created by the CSI driver. Running `terraform destroy` first, while those still existed, would delete the VCN, subnets, and cluster out from under them, risking an orphaned Load Balancer and block volume with no cluster left to manage or clean them up, and no record of them anywhere Terraform looks.

Listing 3.13: The correct teardown order: Kubernetes-managed cloud resources first, Terraform last

```
1 kubectl delete -f k8s\service.yaml
2 kubectl delete -f k8s\deployment.yaml
3 kubectl delete -f k8s\pvc.yaml
4 kubectl delete -f k8s\storageclass.yaml
5 kubectl delete -f k8s\namespace.yaml
6     destroy
```

Deleting the Service first triggers the Cloud Controller Manager to tear down the real Load Balancer and waits for that to complete before the command returns; deleting the PVC, with `reclaimPolicy: Delete` set on its StorageClass, triggers the CSI driver to delete the real underlying block volume. Only once both were confirmed gone did I run `terraform destroy`, at which point it had a clean job: delete exactly the 23 infrastructure resources it actually provisioned, with nothing extra lurking underneath that it had no knowledge of. I walk through both teardown cycles, and the full timing of each, in Chapter 4.

Chapter 4

Deployment Results and Verification

In this chapter, I present the environment as I actually deployed and verified it, not as I designed it on paper. Every name, IP address, CIDR block, timing figure, and error message shown below is taken directly from a live command I ran, or a screenshot I captured, against the real environment, across both build cycles described in Section 1.3.

4.1 The Deployed Network and Cluster Topology

Figure 4.1 shows every resource in the environment I deployed: the VCN, its four subnets, the OKE cluster and its node pool, the application’s PersistentVolumeClaim, and the Kubernetes-provisioned Load Balancer, together with the traffic paths between them.

Table 4.1: CIDR allocation for the deployed VCN.

Network	CIDR block	Purpose
VCN	10.0.0.0/16	Top-level container
tf-lab-oke-vcn-AliYousef		
Endpoint subnet tf-lab-oke-endpoint-subnet-AliYousef	10.0.0.0/28	Kubernetes API endpoint, public
LB subnet tf-lab-oke-lb-subnet-AliYousef	10.0.8.0/24	Public load balancer subnet
Workers subnet tf-lab-oke-workers-subnet-AliYousef	10.0.16.0/20	Worker node VNICs, private
Pods subnet tf-lab-oke-pods-subnet-AliYousef	10.0.32.0/19	VCN-native pod IPs, private, deliberately large

The pods subnet is sized far larger than the other three, a /19 against endpoint’s /28 and

workers' /20, because under VCN-native pod networking (Section 2.2) every pod consumes a real address from this subnet directly, unlike the overlay alternative.

Every subnet's rules are generated from `subnets.tf` via the `subnet` module's `dynamic` blocks (Section 3.3). Tables 4.2–4.5 list every rule I actually applied across all four subnets: this is the authoritative security posture of the environment, not an aspirational diagram.

Table 4.2: Endpoint subnet ingress rules. Egress: Kubernetes API to kubelet (TCP 10250) and to OCI services (TCP 443).

Proto/Port	Source	Purpose
TCP 6443	Workers subnet (10.0.16.0/20)	Worker nodes -> Kubernetes API
TCP 12250	Workers subnet (10.0.16.0/20)	Worker nodes -> OKE control plane services
TCP 6443	0.0.0.0/0	External <code>kubectl</code> access; added only when <code>is_endpoint_public = true</code> , via the <code>concat()</code> pattern in Section 3.5

Table 4.3: Workers subnet ingress rules. Egress: all protocols to 0.0.0.0/0 (image pulls, OCI API calls).

Proto/Port	Source	Purpose
TCP 10250	Endpoint subnet (10.0.0.0/28)	Kubernetes API -> kubelet
All	Pods subnet (10.0.32.0/19)	Pod traffic to worker nodes
All	Workers subnet (10.0.16.0/20)	Worker-to-worker traffic

Table 4.4: Pods subnet ingress rules. Egress: all protocols to 0.0.0.0/0.

Proto/Port	Source	Purpose
All	VCN (10.0.0.0/16)	Pod-to-pod, worker-to-pod, and load balancer health-check traffic

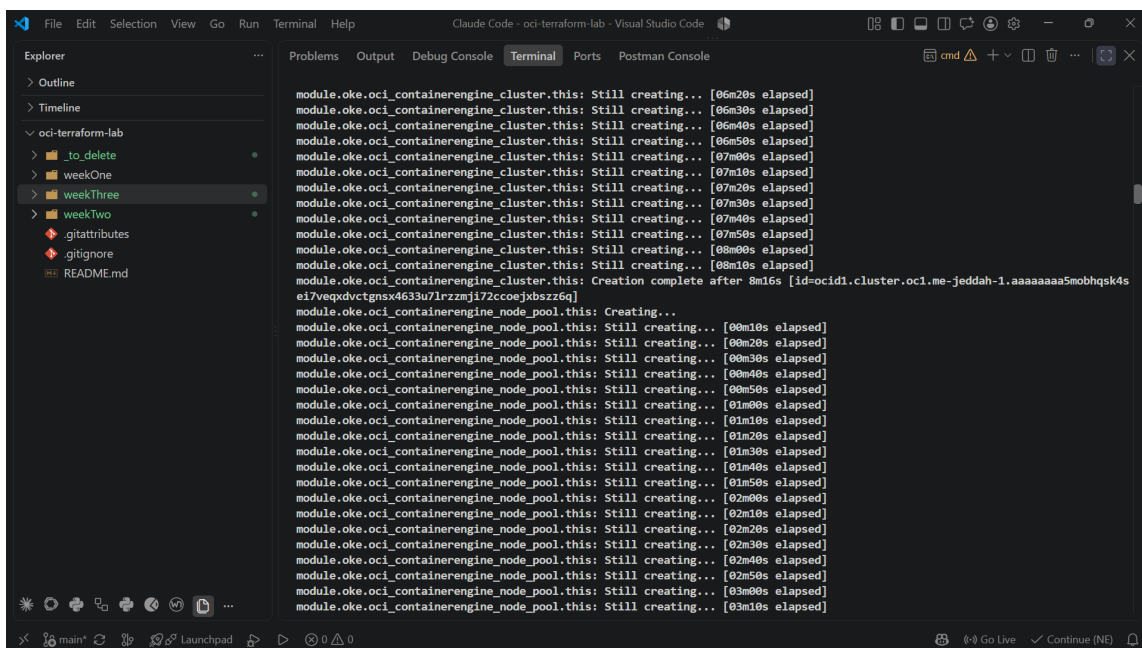
Table 4.5: LB subnet ingress rules. Egress: all protocols to the workers subnet (NodePort health checks and traffic).

Proto/Port	Source	Purpose
TCP 80	0.0.0.0/0	Public HTTP to the load balancer
TCP 443	0.0.0.0/0	Public HTTPS to the load balancer (no HTTPS listener currently configured; see Section 6.3)

The pattern to notice, the same one I established in Week 2: **the workers and pods subnets accept no inbound traffic from outside the VCN at all**. Every rule in Tables 4.3 and 4.4 sources from inside the VCN’s own `10.0.0.0/16`. The only two subnets reachable from the public internet are the load balancer subnet, on 80 and 443, and the endpoint subnet, on 6443, and the latter only because I left `is_endpoint_public` at its lab-convenient default of `true`, a tradeoff I revisit in Section 4.7.

4.2 Two Independent Build Cycles, Verified

I built and fully tore down this entire environment twice. The first cycle ended in a teardown before the application layer was deployed, purely to avoid paying for an idle cluster, node pool, and NAT gateway overnight; the second cycle carried the deployment all the way through to a verified, publicly reachable application before its own teardown. Figure 4.2 is the live `terraform apply` output from the cluster and node pool coming up: the cluster’s control plane reached `Active` after 8 minutes 16 seconds, after which the node pool began its own, separately-timed creation.



```

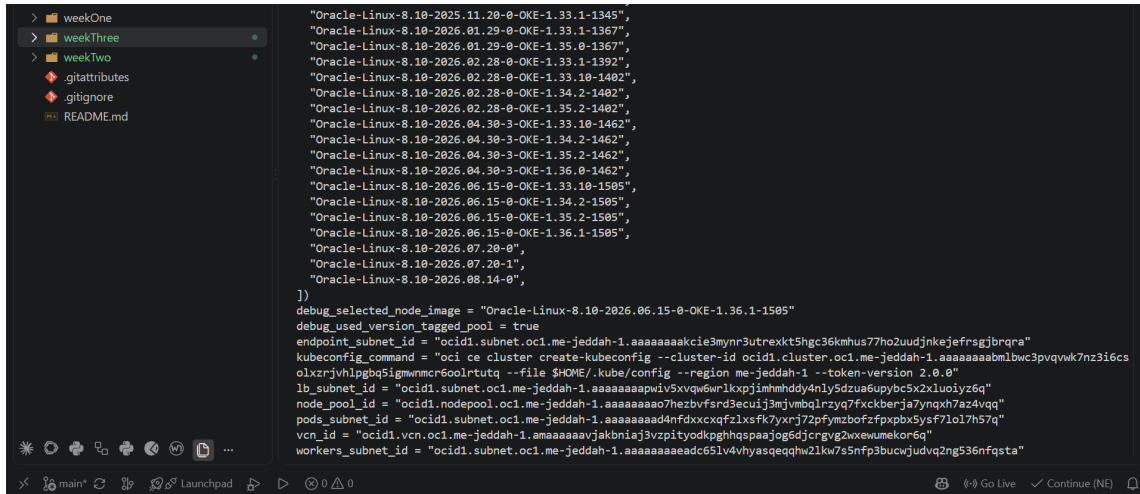
module.oke.oci_containerengine_cluster.this: Still creating... [8m20s elapsed]
module.oke.oci_containerengine_cluster.this: Still creating... [8m30s elapsed]
module.oke.oci_containerengine_cluster.this: Still creating... [8m40s elapsed]
module.oke.oci_containerengine_cluster.this: Still creating... [8m50s elapsed]
module.oke.oci_containerengine_cluster.this: Still creating... [8m59s elapsed]
module.oke.oci_containerengine_cluster.this: Still creating... [9m09s elapsed]
module.oke.oci_containerengine_cluster.this: Still creating... [9m19s elapsed]
module.oke.oci_containerengine_cluster.this: Still creating... [9m29s elapsed]
module.oke.oci_containerengine_cluster.this: Still creating... [9m39s elapsed]
module.oke.oci_containerengine_cluster.this: Still creating... [9m49s elapsed]
module.oke.oci_containerengine_cluster.this: Still creating... [9m59s elapsed]
module.oke.oci_containerengine_cluster.this: Still creating... [10m09s elapsed]
module.oke.oci_containerengine_cluster.this: Still creating... [10m10s elapsed]
module.oke.oci_containerengine_cluster.this: Creation complete after 8m16s [id=ocid1.cluster.oc1.me-jeddah-1.aaaaaaa5mobhqs4s
e17veqxdvctgnsx4633u7lrzmmji72ccoejbsz6q]
module.oke.oci_containerengine_node_pool.this: Creating...
module.oke.oci_containerengine_node_pool.this: Still creating... [00m10s elapsed]
module.oke.oci_containerengine_node_pool.this: Still creating... [00m20s elapsed]
module.oke.oci_containerengine_node_pool.this: Still creating... [00m30s elapsed]
module.oke.oci_containerengine_node_pool.this: Still creating... [00m40s elapsed]
module.oke.oci_containerengine_node_pool.this: Still creating... [00m50s elapsed]
module.oke.oci_containerengine_node_pool.this: Still creating... [01m00s elapsed]
module.oke.oci_containerengine_node_pool.this: Still creating... [01m10s elapsed]
module.oke.oci_containerengine_node_pool.this: Still creating... [01m20s elapsed]
module.oke.oci_containerengine_node_pool.this: Still creating... [01m30s elapsed]
module.oke.oci_containerengine_node_pool.this: Still creating... [01m40s elapsed]
module.oke.oci_containerengine_node_pool.this: Still creating... [01m50s elapsed]
module.oke.oci_containerengine_node_pool.this: Still creating... [02m00s elapsed]
module.oke.oci_containerengine_node_pool.this: Still creating... [02m10s elapsed]
module.oke.oci_containerengine_node_pool.this: Still creating... [02m20s elapsed]
module.oke.oci_containerengine_node_pool.this: Still creating... [02m30s elapsed]
module.oke.oci_containerengine_node_pool.this: Still creating... [02m40s elapsed]
module.oke.oci_containerengine_node_pool.this: Still creating... [02m50s elapsed]
module.oke.oci_containerengine_node_pool.this: Still creating... [03m00s elapsed]
module.oke.oci_containerengine_node_pool.this: Still creating... [03m10s elapsed]

```

Figure 4.2: `terraform apply`, captured live: `module.oke.oci_containerengine_cluster.this`: Creation complete after 8m16s, immediately followed by the node pool resource beginning its own, separately-timed `Creating...` phase.

Figure 4.3 is the same build’s final Terraform outputs, showing the real OCIDs Terraform recorded for every major resource, the exact `oci ce cluster create-kubeconfig` command generated from those OCIDs, and the confirmed worker image selection,

Oracle-Linux-8.10-2026.06.15-0-OKE-1.36.1-1505, matching Section 3.6's iteration-3 logic exactly.



```

> weekOne
> weekThree
> weekTwo
> .gitattributes
> .gitignore
> README.md

"Oracle-Linux-8.10-2025.11.20-0-OKE-1.33.1-1345",
"Oracle-Linux-8.10-2026.01.29-0-OKE-1.33.1-1367",
"Oracle-Linux-8.10-2026.01.29-0-OKE-1.35.0-1367",
"Oracle-Linux-8.10-2026.02.28-0-OKE-1.33.1-1392",
"Oracle-Linux-8.10-2026.02.28-0-OKE-1.33.10-1402",
"Oracle-Linux-8.10-2026.02.28-0-OKE-1.34.2-1402",
"Oracle-Linux-8.10-2026.02.28-0-OKE-1.35.2-1402",
"Oracle-Linux-8.10-2026.04.30-3-OKE-1.33.10-1462",
"Oracle-Linux-8.10-2026.04.30-3-OKE-1.34.2-1462",
"Oracle-Linux-8.10-2026.04.30-3-OKE-1.35.2-1462",
"Oracle-Linux-8.10-2026.04.30-3-OKE-1.36.0-1462",
"Oracle-Linux-8.10-2026.06.15-0-OKE-1.33.10-1505",
"Oracle-Linux-8.10-2026.06.15-0-OKE-1.34.2-1505",
"Oracle-Linux-8.10-2026.06.15-0-OKE-1.35.2-1505",
"Oracle-Linux-8.10-2026.06.15-0-OKE-1.36.1-1505",
"Oracle-Linux-8.10-2026.07.20-0",
"Oracle-Linux-8.10-2026.07.20-1",
"Oracle-Linux-8.10-2026.08.14-0",
})
debug_selected_node_image = "Oracle-Linux-8.10-2026.06.15-0-OKE-1.36.1-1505"
debug_used_version_tagged_pool = true
endpoint_subnet_id = "ocid1.subnet.oc1.me-jeddah-1.aaaaaaakcie3mynr3utrexktShgc36kmhus77ho2uudjnkjejrsgjbrqra"
kubeconfig_command = "oci ce cluster create-kubeconfig --cluster-id ocid1.cluster.oc1.me-jeddah-1.aaaaaaabmlbwc3pvqvwk7nz3ifcs
olxjrjvhlgpbq5igmmmc6oolrtutq --file $HOME/.kube/config --region me-jeddah-1 --token-version 2.0.0"
lb_subnet_id = "ocid1.subnet.oc1.me-jeddah-1.aaaaaaaapwivSxvq6wrlkxpjlmhmdy4nly5dzua6upybcs5x2xluoyz6q"
node_pool_id = "ocid1.nodepool.oc1.me-jeddah-1.aaaaaaaao7hezbfvfrd3ecuij3mjmbqlr2yq7fxcckberja7ynqxh7az4vqg"
pods_subnet_id = "ocid1.subnet.oc1.me-jeddah-1.aaaaaaaad4nfdxcxqfz1xsfk7yxrj72pfymzbofzfpzpxbxs5ysf71ol7h57q"
vcn_id = "ocid1.vcn.oc1.me-jeddah-1.aaaaaaaavjakbniaj3vzpityodkpgghqspaa3og6djcrvg2wxxewumekor6q"
workers_subnet_id = "ocid1.subnet.oc1.me-jeddah-1.aaaaaaaeadc651v4vhyasqeqqhw21kw7s5nfp3bucvjudvq2ng536nfqsta"

```

Figure 4.3: terraform apply, captured live: final outputs including `cluster_id`, `node_pool_id`, the four subnet OCIDs, the ready-to-run `kubeconfig_command`, and the diagnostic `debug_selected_node_image/debug_used_version_tagged_pool` outputs confirming the version-tagged branch of the selection logic was the one actually used.

Both cycles applied and destroyed the same 23 Terraform-managed resources, with zero errors on either apply and zero errors on either destroy; I return to the destroy side, and its timing, in Section 4.6.

4.3 Reaching the Cluster: An IAM Gap in the OCI CLI's Setup Flow

Generating a working `kubeconfig` needs the OCI CLI, and the CLI's own interactive setup, triggered automatically the first time I ran `oci ce cluster create-kubeconfig` with no existing `~/.oci/config`, tried to authenticate through a browser and then self-provision a brand-new API signing key on my behalf, via an `upload_api_key` call. Figure 4.4 is the browser completing that authentication step.



Figure 4.4: The OCI CLI's interactive setup flow: browser-based login completed successfully, redirecting back to the local bootstrap process on `localhost:8181`.

The key self-provisioning step that followed did not succeed, shown in Figure 4.5.

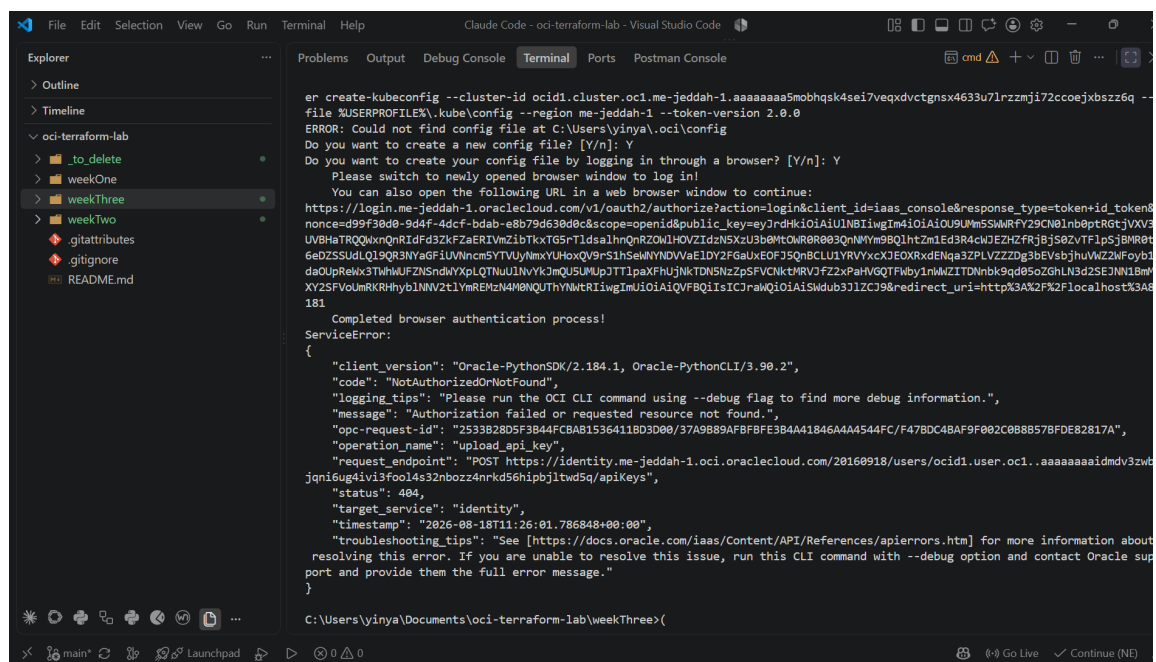


Figure 4.5: The OCI CLI, captured live: after completing browser authentication, the setup flow's `upload_api_key` call against the Identity service returns `"code": "NotAuthorizedOrNotFound"`, HTTP 404.

OCI deliberately returns 404 rather than 403 for authorization failures, so a client cannot distinguish “this does not exist” from “you cannot see this.” This is the same class of environment gap I hit with the Bastion service’s IAM policy in Week 2: this identity is authorized to

use an API key that already exists and was provisioned through the proper channel, the same key pair already sitting in `terraform.tfvars` and already working for every `terraform apply` and `destroy` all week, but is not authorized to self-provision a *new* one through this interactive flow. Rather than treat this as something to work around by requesting a wider IAM policy, out of scope for this lab, I skipped the interactive flow entirely and hand-wrote `~/.oci/config`, reusing the tenancy OCID, user OCID, fingerprint, and key file path already present in `terraform.tfvars`. With that file in place, `create-kubeconfig` succeeded immediately, producing only two advisory, non-blocking warnings about the generated config and key file's Windows permissions being broader than the CLI would prefer, visible at the top of Figure 4.8 and repeated throughout this chapter's later figures; they never blocked any command from completing.

The generated `kubeconfig` authenticates via a Kubernetes `exec` credential plugin that shells out to the `oci` executable to mint a short-lived token on demand. On this machine, that executable was not on the system `PATH`, so the first `kubectl get nodes` failed with `exec: executable oci not found`; I fixed this by patching only the `command: oci` line inside the generated `kubeconfig` to the executable's full path, rather than modifying the system-wide `PATH`, which risks silently truncating an already-long existing value. `kubectl get nodes` then succeeded, confirming both worker nodes `Ready`.

4.4 Two Live Application-Layer Bugs, Diagnosed and Fixed

Every Kubernetes manifest in Section 3.7 reflects its *final*, working state. Getting there needed diagnosing and fixing two real, live bugs, neither of which was visible from reading the manifests alone before applying them.

4.4.1 CRI-O Rejects Unqualified Image References

Applying the Deployment manifest for the first time left the pod stuck, and `kubectl describe pod` surfaced the actual cause on the `nginx` container specifically, shown in Figure 4.6.

```

State:      Terminated
Reason:     Completed
Exit Code:  0
Started:    Tue, 18 Aug 2026 14:34:10 +0300
Finished:   Tue, 18 Aug 2026 14:34:10 +0300
Ready:      True
Restart Count: 0
Environment:
  NODE_NAME: (v1:spec.nodeName)
Mounts:
  /data from data (rw)
  /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-xnd9k (ro)
Containers:
  nginx:
    Container ID:
    Image:
    Image ID:    nginx:1.27-alpine
    Port:        80/TCP
    Host Port:   0/TCP
    State:       Waiting
      Reason:    ImageInspectError
    Ready:       False
    Restart Count: 0
    Limits:
      cpu:       250m
      memory:    256Mi
    Requests:
      cpu:       100m
      memory:    128Mi
    Liveness:    http-get http://:80/index.html delay=10s timeout=1s period=10s #success=1 #failure=3
    Readiness:   http-get http://:80/index.html delay=3s timeout=1s period=5s #success=1 #failure=3
    Environment: <none>
    Mounts:

```

Figure 4.6: `kubectl describe pod`, captured live: the `nginx` container, image `nginx:1.27-alpine`, State: `Waiting`, Reason: `ImageInspectError`.

Scrolling further down the same `describe pod` output, shown in Figure 4.7, confirmed the node's container runtime directly: the `initContainer`'s `Container ID` is prefixed `cri-o://`, and its `Image ID` resolves to a fully-qualified `docker.io/library/busybox` digest, since that image happened to already be cached locally on the node, sidestepping the same resolution rule that rejected the unqualified `nginx` reference.

```

IP: 10.0.33.104
Controlled By: ReplicaSet/demo-app-788d46f774
Init Containers:
  write-index:
    Container ID: cri-o://42c31e2564332d5219d1e98025f8ead8cd4e870881958accdbda1e1790fd1730
    Image: busybox:1.36
    Image ID: docker.io/library/busybox@sha256:73aaf090f3d85aa34ee199857f03fa3a95c8ede2ff4d4cc2c2db5b94e566b11662
    Port: <none>
    Host Port: <none>
    Command:
      sh
      -c
      cat <<EOF > /data/index.html
      <html>
      <head><title>Ejada Week 3 - OKE Demo</title></head>
      <body style="font-family: sans-serif;">
      <h1>Ejada Internship - Week 3 OKE Lab</h1>
      <p>Served by pod: $(hostname)</p>
      <p>Node: $NODE_NAME</p>
      <p>Rendered at container start: $(date -u)</p>
      <p>This page is being read from a PersistentVolumeClaim backed
        by an OCI block volume, attached via the CSI driver.</p>
      </body>
      </html>
      EOF
    State: Terminated
    Reason: Completed
    Exit Code: 0
    Started: Tue, 18 Aug 2026 14:34:10 +0300
    Finished: Tue, 18 Aug 2026 14:34:10 +0300
    Ready: True
    Restart Count: 0
    Environment:

```

Figure 4.7: `kubectl describe pod`, captured live: the `initContainer`’s `Container ID`: `cri-o://...` identifies the node’s runtime as CRI-O, and its full command shows the exact here-document script that renders `index.html`, matching `k8s/deployment.yaml` in Section 3.7.2.

This node image runs CRI-O with short-name resolution mode set to enforcing (Section 2.5): an unqualified reference like `nginx:1.27-alpine` is rejected outright, with the message “*short name mode is enforcing, but image name ... returns ambiguous list*”, rather than resolved against a guessed registry. Fully qualifying both image references with an explicit `docker.io/library/` prefix, already shown as the current, final state of `deployment.yaml` in Section 3.7.2, resolved the error on the next `kubectl apply`.

4.4.2 A ReadWriteOnce Volume Deadlocks the Default Rollout Strategy

Fixing the image reference and reapplying triggered a second, unrelated failure. The new pod’s `initContainer` sat at `Init:0/1` for roughly thirty minutes with no visible progress, while the previous, still-broken pod remained present rather than being torn down. Figure 4.8 is that state, captured live.

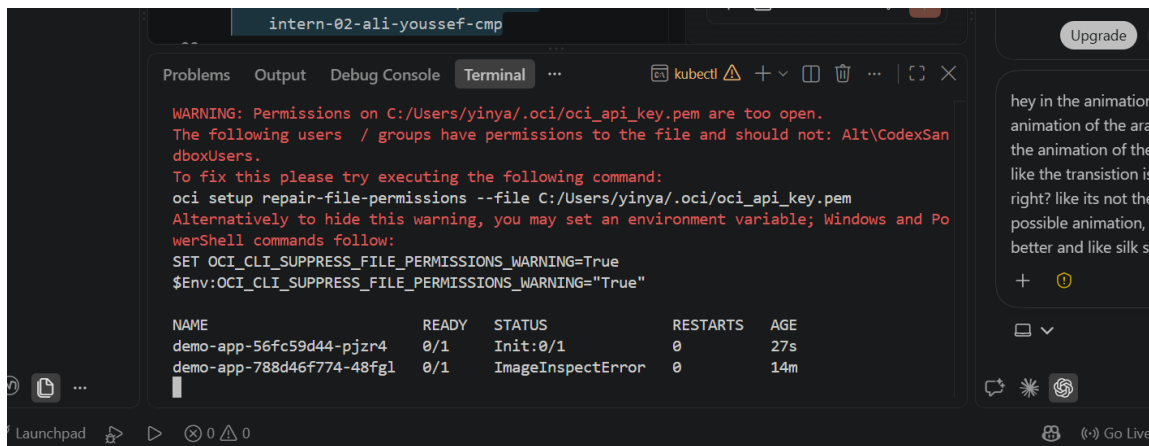


Figure 4.8: `kubectl get pods`, captured live: `demo-app-56fc59d44-pjzr4` at `Init:0/1`, `demo-app-788d46f774-48fgl` still present at `ImageInspectError`, neither progressing.

The cause is the default `RollingUpdate` strategy interacting badly with the `PersistentVolumeClaim`'s `ReadWriteOnce` access mode (Section 2.3): `RollingUpdate` brings the replacement pod up *before* tearing down the old one, to achieve zero-downtime rollouts, but the replacement's `initContainer` cannot attach the volume while the old pod's node still holds it, and the old pod will not be torn down until the rollout considers the replacement healthy, a genuine circular deadlock with no way to resolve on its own. I unblocked the immediate rollout by manually deleting the stuck old pod, which freed the volume immediately, and fixed it durably by adding `strategy: type: Recreate` to `deployment.yaml` (Section 3.7.2), which tears down all existing pods before creating any replacement, matching what the storage constraint actually requires.

A confusing follow-up appeared shortly after: a second rollout began, producing a new pod carrying the *original*, pre-any-fix template hash, which hit the exact same `ImageInspectError` again. Figure 4.9 is the full `kubectl get pods -w` sequence I watched live through the whole incident, and is worth reading closely: it shows the correctly-fixed pod, `demo-app-56fc59d44-pjzr4`, already sitting at `Init:0/1` for eighteen minutes when a second, differently-hashed pod, `demo-app-788d46f774-6mhcx`, appears, reaches `PodInitializing` within under a minute, and then hits `ImageInspectError` at 56 seconds into its own life, the same CRI-O short-name failure from Section 4.4.1, on the old, unqualified image reference that hash still carries.

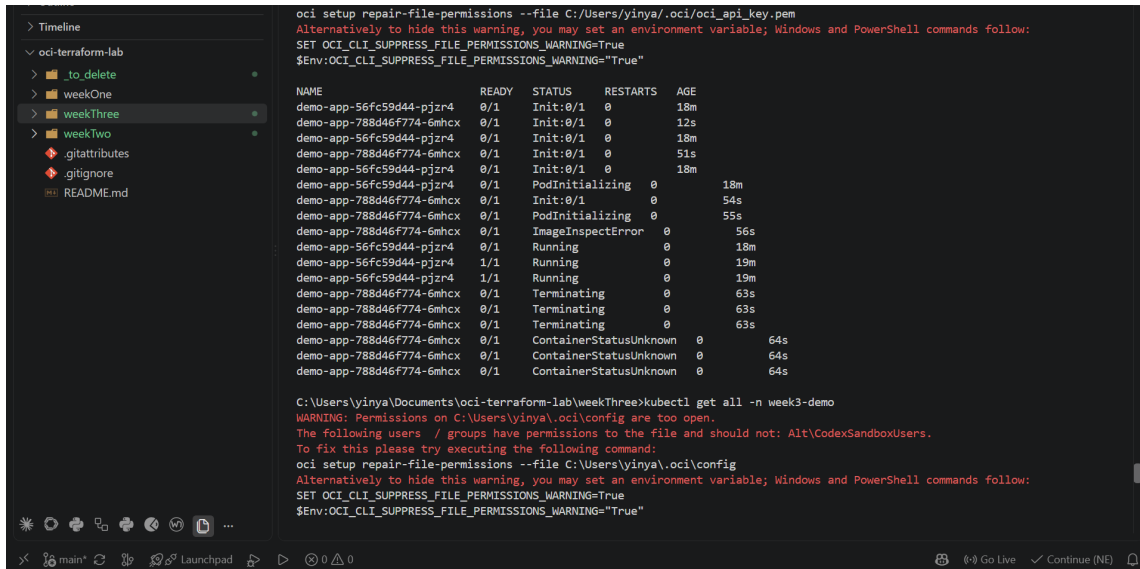


Figure 4.9: `kubectl get pods -w`, captured live across the whole incident: `pjzr4` (hash `56fc59d44`, the correctly-fixed pod) stuck at `Init:0/1` for 18 minutes; `6mhcX` (hash `788d46f774`, the stale, pre-fix pod) appearing, reaching `PodInitializing`, then `ImageInspectError` within under a minute; `pjzr4` then reaching `Running 1/1`; `6mhcX` then `Terminating` and `ContainerStatusUnknown`.

My working explanation, not fully confirmable after the fact but consistent with every piece of evidence in Figure 4.9, is that a `kubectl apply` ran at some point against a momentarily stale copy of `deployment.yaml` on disk. Kubernetes computes a Deployment’s ReplicaSet identity from a hash of its pod template, so applying an older template content, even briefly, causes Kubernetes to reuse and scale back up the pre-existing ReplicaSet matching that older hash rather than creating a new one. This posed no real risk to the already-healthy pod: Kubernetes’ default Deployment behavior does not tear down a healthy pod to make way for a new pod that is not yet passing its own readiness checks, which is exactly what Figure 4.9 shows, `pjzr4` reaching `Running 1/1` independently of whatever was happening to `6mhcX`. The situation resolved on its own once a subsequent, genuinely correct `kubectl apply` took effect: Kubernetes scaled the stale ReplicaSet back to zero and left the correct one as the sole active one, confirmed in Section 4.5.

One further, genuinely instructive detail surfaced while this was still unresolved. A browser request against the load balancer’s public IP, shown in Figure 4.10, returned content stamped with `6mhcX`’s own hostname, even though that pod’s main `nginx` container never reached `Running`.

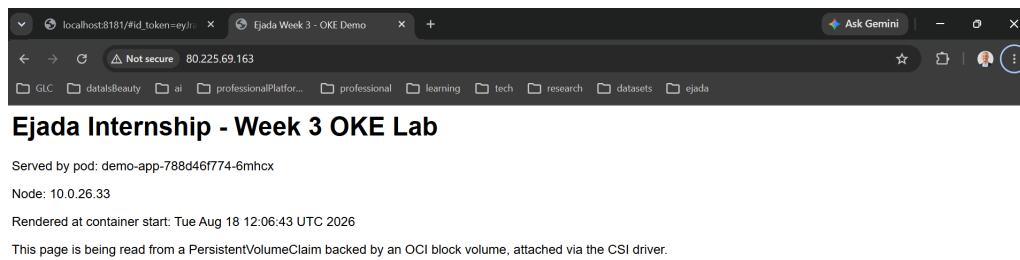


Figure 4.10: A live browser request against `80.225.69.163`, captured mid-incident: the page renders “*Served by pod: demo-app-788d46f774-6mhcx*”, even though that pod’s `nginx` container never left `ImageInspectError`.

This is not a data-integrity bug; it is a direct, if initially surprising, consequence of the `PersistentVolumeClaim` being shared, node-scoped storage. `6mhcx`’s `initContainer`, using the already-fixed `busybox` image, completed successfully even though its sibling `nginx` container never started, and because `6mhcx` and the genuinely healthy `pjzr4` were scheduled to the same worker node, and therefore shared the same underlying mounted volume, `6mhcx`’s `initContainer` overwrote `/data/index.html` with its own hostname and timestamp before failing. `pjzr4`’s `nginx` container, still running, then simply served whatever currently existed on disk. Once the stale `ReplicaSet` scaled back to zero, no pod could overwrite that file again, and Section 4.5’s browser check reflects the correct pod, as it should.

4.5 The Application Verified End to End

Once both bugs above were resolved, I confirmed every layer of the deployment independently. Figure 4.11 confirms the `PersistentVolumeClaim` reached `Bound` at the requested size against the correct `StorageClass`.

```

Alternatively to hide this warning, you may set an environment variable; Windows and PowerShell commands follow:
SET OCI_CLI_SUPPRESS_FILE_PERMISSIONS_WARNING=True
$Env:OCI_CLI_SUPPRESS_FILE_PERMISSIONS_WARNING="True"

WARNING: Permissions on C:/Users/yinya/.oci/oci_api_key.pem are too open.
The following users / groups have permissions to the file and should not: Alt\CodexSandboxUsers.
To fix this please try executing the following command:
oci setup repair-file-permissions --file C:/Users/yinya/.oci/oci_api_key.pem
Alternatively to hide this warning, you may set an environment variable; Windows and PowerShell commands follow:
SET OCI_CLI_SUPPRESS_FILE_PERMISSIONS_WARNING=True
$Env:OCI_CLI_SUPPRESS_FILE_PERMISSIONS_WARNING="True"

NAME              STATUS    VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCL
---              -
demo-app-data     Bound    csi-54b76a93-5875-4131-8d10-b29b7f0e05fc  50Gi        RWO            week3-oci-bv    <unset>

C:\Users\yinya\Documents\oci-terraform-lab\weekThree\kubectl get pvc -n week3-demo
demo-app-data     Bound    csi-54b76a93-5875-4131-8d10-b29b7f0e05fc  50Gi        RWO            week3-oci-bv    <unset>

```

Figure 4.11: `kubectl get pvc -n week3-demo`, captured live: `demo-app-data`, Bound, 50Gi, RWO, StorageClass `week3-oci-bv`.

Figure 4.12 is the complete, final state of every object in the `week3-demo` namespace: exactly one pod, 1/1 Running with zero restarts; the Service, type: `LoadBalancer`, with its real external IP populated; the Deployment at 1/1 ready and available; and, critically, the two ReplicaSets confirming the incident in Section 4.4.2 resolved cleanly, the correct hash at 1/1/1 and the stale hash at 0/0/0 rather than deleted outright, which is normal Deployment revision-history bookkeeping, not lingering evidence of a problem.

```

$Env:OCI_CLI_SUPPRESS_FILE_PERMISSIONS_WARNING="True"

WARNING: Permissions on C:\Users\yinya\.oci\oci_api_key.pem are too open.
The following users / groups have permissions to the file and should not: Alt\CodexSandboxUsers.
To fix this please try executing the following command:
oci setup repair-file-permissions --file C:\Users\yinya\.oci\oci_api_key.pem
Alternatively to hide this warning, you may set an environment variable; Windows and PowerShell commands follow:
SET OCI_CLI_SUPPRESS_FILE_PERMISSIONS_WARNING=True
$Env:OCI_CLI_SUPPRESS_FILE_PERMISSIONS_WARNING="True"

NAME                                READY   STATUS    RESTARTS   AGE
pod/demo-app-56fc59d44-pjzr4        1/1     Running   0           28m

NAME                                TYPE          CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
service/demo-app                    LoadBalancer  10.96.107.47  80.225.69.163  80:30875/TCP     42m

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/demo-app            1/1     1             1           43m

NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/demo-app-56fc59d44  1         1         1       28m
replicaset.apps/demo-app-788d46f774 0         0         0       43m

C:\Users\yinya\Documents\oci-terraform-lab\weekThree>kubectl get pvc -n week3-demo
WARNING: Permissions on C:\Users\yinya\.oci\config are too open.
The following users / groups have permissions to the file and should not: Alt\CodexSandboxUsers.
To fix this please try executing the following command:
oci setup repair-file-permissions --file C:\Users\yinya\.oci\config
Alternatively to hide this warning, you may set an environment variable; Windows and PowerShell commands follow:
SET OCI_CLI_SUPPRESS_FILE_PERMISSIONS_WARNING=True
$Env:OCI_CLI_SUPPRESS_FILE_PERMISSIONS_WARNING="True"

WARNING: Permissions on C:\Users\yinya\.oci\oci_api_key.pem are too open.
The following users / groups have permissions to the file and should not: Alt\CodexSandboxUsers.

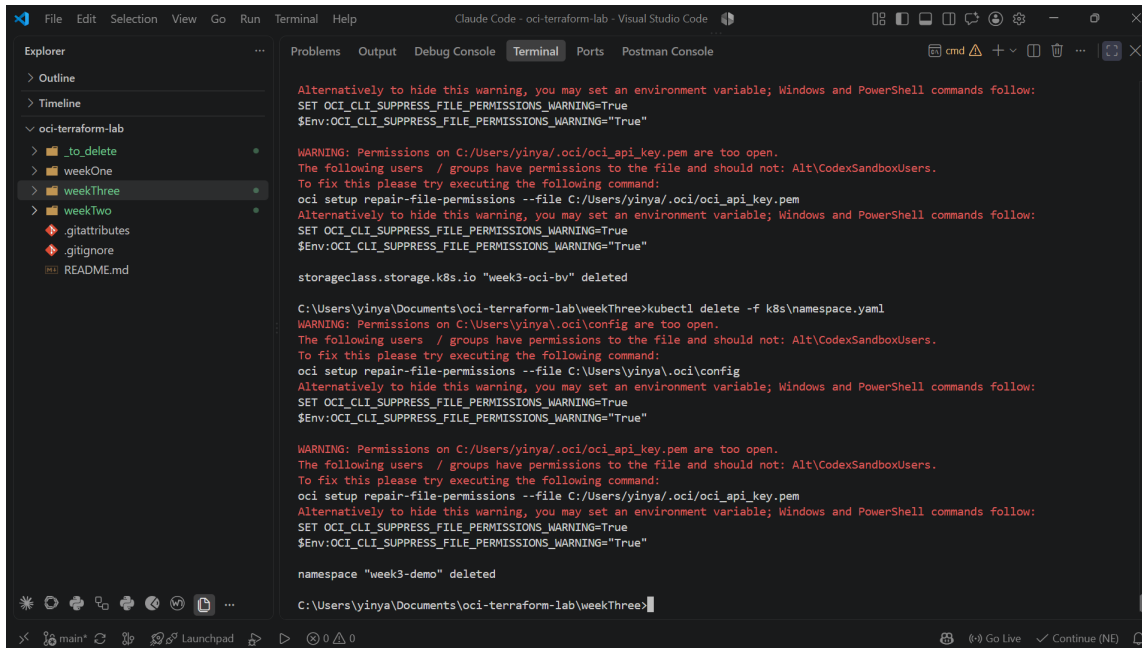
```

Figure 4.12: `kubectl get all -n week3-demo`, captured live, final verified state: pod `demo-app-56fc59d44-pjzr4` 1/1 Running, 0 restarts; Service `demo-app` LoadBalancer, cluster IP 10.96.107.47, external IP 80.225.69.163, port 80:30875/TCP; Deployment 1/1; ReplicaSet `demo-app-56fc59d44` at 1/1/1, ReplicaSet `demo-app-788d46f774` at 0/0/0.

Finally, I opened `http://80.225.69.163` directly in an ordinary browser, with no VPN or bastion hop of any kind, and confirmed the same proof-of-life content Figure 4.10 showed earlier, now correctly attributed to the healthy, 1/1 Running pod, closing the loop from a Terraform-provisioned cluster, through a Kubernetes-provisioned load balancer and block volume, to a real HTTP response a browser can render.

4.6 Environment Teardown, Executed Twice

I deleted every Kubernetes-managed object before ever running `terraform destroy`, in the order specified in Section 3.8. Figure 4.13 confirms the StorageClass and Namespace deletions completing cleanly, after the Service, Deployment, and PVC had already been removed in the preceding steps of the same session.



```

Alternatively to hide this warning, you may set an environment variable; Windows and PowerShell commands follow:
SET OCI_CLI_SUPPRESS_FILE_PERMISSIONS_WARNING=True
$Env:OCI_CLI_SUPPRESS_FILE_PERMISSIONS_WARNING="True"

WARNING: Permissions on C:\Users\yinya\.oci\oci_api_key.pem are too open.
The following users / groups have permissions to the file and should not: Alt\CodexSandboxUsers.
To fix this please try executing the following command:
oci setup repair-file-permissions --file C:\Users\yinya\.oci\oci_api_key.pem
Alternatively to hide this warning, you may set an environment variable; Windows and PowerShell commands follow:
SET OCI_CLI_SUPPRESS_FILE_PERMISSIONS_WARNING=True
$Env:OCI_CLI_SUPPRESS_FILE_PERMISSIONS_WARNING="True"

storageclass.storage.k8s.io "week3-oci-bv" deleted

C:\Users\yinya\Documents\oci-terraform-lab\weekThree>kubectl delete -f k8s\namespace.yaml
WARNING: Permissions on C:\Users\yinya\.oci\config are too open.
The following users / groups have permissions to the file and should not: Alt\CodexSandboxUsers.
To fix this please try executing the following command:
oci setup repair-file-permissions --file C:\Users\yinya\.oci\config
Alternatively to hide this warning, you may set an environment variable; Windows and PowerShell commands follow:
SET OCI_CLI_SUPPRESS_FILE_PERMISSIONS_WARNING=True
$Env:OCI_CLI_SUPPRESS_FILE_PERMISSIONS_WARNING="True"

WARNING: Permissions on C:\Users\yinya\.oci\oci_api_key.pem are too open.
The following users / groups have permissions to the file and should not: Alt\CodexSandboxUsers.
To fix this please try executing the following command:
oci setup repair-file-permissions --file C:\Users\yinya\.oci\oci_api_key.pem
Alternatively to hide this warning, you may set an environment variable; Windows and PowerShell commands follow:
SET OCI_CLI_SUPPRESS_FILE_PERMISSIONS_WARNING=True
$Env:OCI_CLI_SUPPRESS_FILE_PERMISSIONS_WARNING="True"

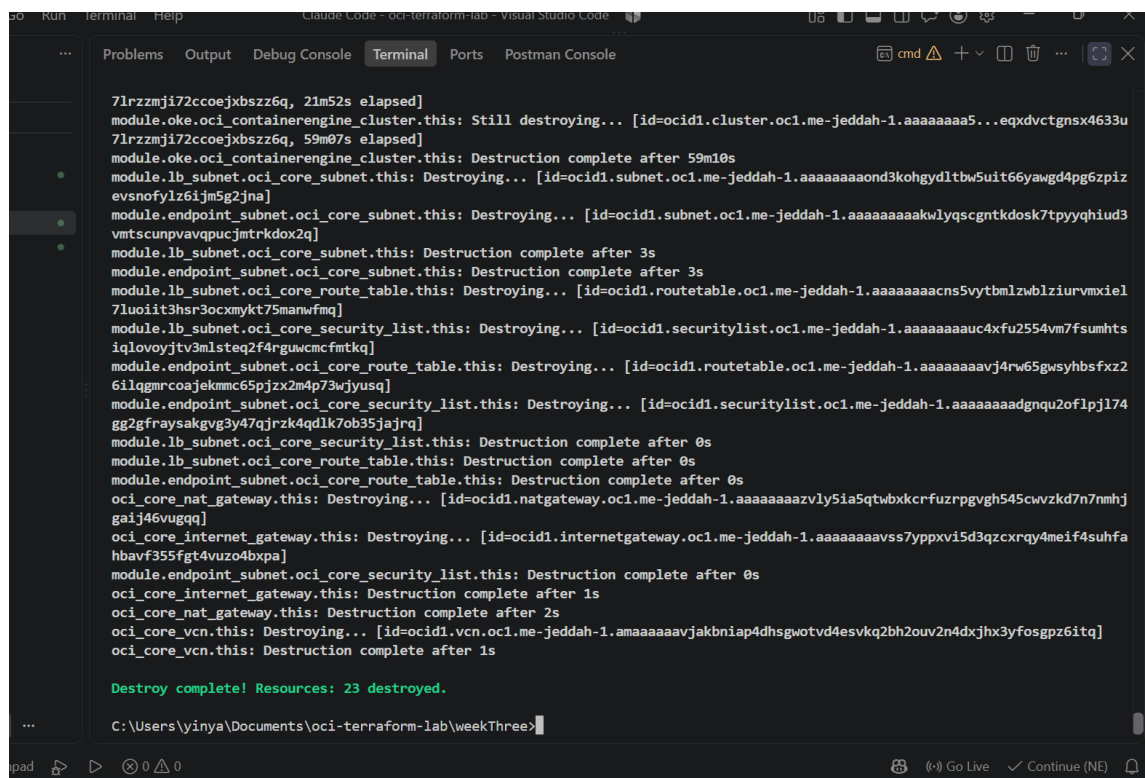
namespace "week3-demo" deleted

C:\Users\yinya\Documents\oci-terraform-lab\weekThree>

```

Figure 4.13: `kubectl delete`, captured live: `storageclass.storage.k8s.io "week3-oci-bv"` deleted, followed by namespace `"week3-demo"` deleted, the final two steps of the Kubernetes-layer teardown before `terraform destroy` ran.

With no Kubernetes-managed cloud resources left outstanding, `terraform destroy` then had a clean job. Figure 4.14 is the very end of that run: the cluster's managed control plane took 59 minutes 10 seconds to finish destroying, considerably longer than the 8 minutes 16 seconds it took to create in Section 4.2, an asymmetry that reflects OCI's own OKE control-plane deprovisioning process rather than anything Terraform itself is doing slowly; I did not interrupt it. Every remaining resource, the subnets, route tables, security lists, gateways, and finally the VCN itself, destroyed in seconds once the cluster was gone, and the run ended with all 23 resources confirmed destroyed.



```

7lrzzmji72ccoejxbszz6q, 21m52s elapsed]
module.oke.oci_containerengine_cluster.this: Still destroying... [id=ocid1.cluster.oc1.me-jeddah-1.aaaaaaa5...eqxdvctgnsx4633u
7lrzzmji72ccoejxbszz6q, 59m07s elapsed]
module.oke.oci_containerengine_cluster.this: Destruction complete after 59m10s
module.lb_subnet.oci_core_subnet.this: Destroying... [id=ocid1.subnet.oc1.me-jeddah-1.aaaaaaaond3kohgydltbw5uit66yawgd4pg6zpiz
evsnofylz6ijm5g2jna]
module.endpoint_subnet.oci_core_subnet.this: Destroying... [id=ocid1.subnet.oc1.me-jeddah-1.aaaaaaaakwlyqscgntkdosk7tpyyqhiud3
vmtscunpvavqucjmrkdox2q]
module.lb_subnet.oci_core_subnet.this: Destruction complete after 3s
module.endpoint_subnet.oci_core_subnet.this: Destruction complete after 3s
module.lb_subnet.oci_core_route_table.this: Destroying... [id=ocid1.routetable.oc1.me-jeddah-1.aaaaaaaacs5vytblzwbzliurvmxiel
7luoit3hsr3ocxmykt75manwfmq]
module.lb_subnet.oci_core_security_list.this: Destroying... [id=ocid1.securitylist.oc1.me-jeddah-1.aaaaaaauc4xfu2554vm7fsumhts
iqlovoyjtv3mlsteq2f4rguwcmtkq]
module.endpoint_subnet.oci_core_route_table.this: Destroying... [id=ocid1.routetable.oc1.me-jeddah-1.aaaaaaaavj4rw65gwsyhsfxfz2
6ilqgmrcoaajekmnc65pjzx2m4p73wjyusq]
module.endpoint_subnet.oci_core_security_list.this: Destroying... [id=ocid1.securitylist.oc1.me-jeddah-1.aaaaaaaadgnqu2oflpj174
gg2gfraysakvg3y47qjrz4qdlk7ob35jaqrq]
module.lb_subnet.oci_core_security_list.this: Destruction complete after 0s
module.lb_subnet.oci_core_route_table.this: Destruction complete after 0s
module.endpoint_subnet.oci_core_route_table.this: Destruction complete after 0s
oci_core_nat_gateway.this: Destroying... [id=ocid1.natgateway.oc1.me-jeddah-1.aaaaaaaazvly5ia5qtwbxbkcrfuzrpgvgh545cwzkd7n7nmhj
gaij46vugqq]
oci_core_internet_gateway.this: Destroying... [id=ocid1.internetgateway.oc1.me-jeddah-1.aaaaaaaavss7yppxvi5d3qzcxrqy4meif4suhfa
hbavf355fgt4vuzo4bxpa]
module.endpoint_subnet.oci_core_security_list.this: Destruction complete after 0s
oci_core_internet_gateway.this: Destruction complete after 1s
oci_core_nat_gateway.this: Destruction complete after 2s
oci_core_vcn.this: Destroying... [id=ocid1.vcn.oc1.me-jeddah-1.aaaaaaaavjakbniap4dhsgwotvd4esvkq2bh2ouv2n4dxjhx3yfospz6itq]
oci_core_vcn.this: Destruction complete after 1s

Destroy complete! Resources: 23 destroyed.

C:\Users\Yinya\Documents\oci-terraform-lab\weekThree>

```

Figure 4.14: terraform destroy, captured live: module.oke.oci_containerengine_cluster.this: Destruction complete after 59m10s, followed by every remaining resource destroying in seconds, ending in Destroy complete! Resources: 23 destroyed.

4.7 Risk Analysis

Table 4.6: Risks identified in the current configuration.

Risk	Likelihood	Impact	Mitigation or Monitoring Indicator
Single application replica, a consequence of the RWO PersistentVolumeClaim	High (by design)	Medium	Acceptable for a demo; split into a storage-owning tier and a separately scalable stateless tier before any real load, per Section 6.3
<code>is_endpoint_public</code> left at its lab-convenient default of <code>true</code>	High	Medium	The endpoint's security list still restricts port 6443 to a VCN-internal source plus the public rule; narrow to known operator IPs before longer-lived use
Local Terraform state, no locking	Medium	High	Only matters once a second operator is involved; migrate to a remote OCI Object Storage backend before team use
Worker image selection depends on OCI continuing to publish version-tagged, non-GPU, non-ARM images matching the current filter pattern	Low	High	The fallback branch in Section 3.6 degrades to the newest overall candidate if no version-tagged image exists; monitor <code>debug_used_version_tagged_pool</code> on future applies
Bare HTTP load balancer, no TLS termination	Medium	Medium	Acceptable for a lab; add an HTTPS listener with a managed certificate before any real traffic

4.8 Key Unknowns

Several questions remain open for me because they fall outside this lab's scope, not because evidence is missing. I have not exercised the deployment under concurrent multi-operator Terraform runs, so I have identified the local-state risk above without observing it in practice. I ran no load testing against the load balancer or the application, so I do not know how either behaves above the lab-scale 10 Mbps flexible bandwidth ceiling, or under a genuinely concurrent request volume against a single-replica backend. I did not test node or availability-domain failure, so I do not know how the deployment recovers if the specific node holding the PersistentVolumeClaim's volume becomes unavailable. Finally, the worker image selection logic in Section 3.6 has been verified correct against two independent live candidate lists, captured roughly a day apart, but I do not know how it behaves against a candidate list structured differ-

ently from either of those two, for example one where no image at all survives the architecture and variant exclusions.

Chapter 5

Conclusions

5.1 Main Conclusion

The environment I built and documented here is a managed OKE cluster, provisioned from two reusable Terraform modules, running a real Kubernetes application with genuine persistent, block-volume-backed storage and genuine public exposure through a Kubernetes-provisioned load balancer. That answers the objective I set out in Section 1.2 directly: **the cluster runs a real, verifiable application end to end**, confirmed by a live browser request against the load balancer’s public IP returning content rendered by the correct, healthy pod, and **the Terraform configuration that provisions it is genuinely reusable**, demonstrated by rebuilding the entire environment a second time, independently, from the same code, with a single `terraform apply`.

5.2 Strongest Supporting Evidence

Four pieces of evidence carry the most weight for me:

- **Two independent, clean build cycles.** Both applies added the same 23 resources with zero errors, and both destroys removed them cleanly, confirming the configuration is genuinely reproducible rather than having worked once by chance.
- **The live browser request.** Figure 4.12 and the browser check that followed it confirmed the whole path, load balancer, Service, Deployment, pod, and PersistentVolumeClaim, works together, not merely that each resource exists in isolation.
- **The worker-image selection logic, verified independently before being trusted.** I did not treat the third iteration as correct on faith after the first two failed; I reimplemented its filter and sort logic standalone against a captured candidate list and confirmed the result

before sending it back against live infrastructure.

- **Two real bugs, diagnosed to their actual root cause and fixed durably, not worked around.** The CRI-O short-name rejection and the ReadWriteOnce rollout deadlock both trace to a specific, identifiable mechanism, not a guess-and-check fix; both fixes are now permanent parts of `deployment.yaml`, not one-time manual interventions.

5.3 Biggest Uncertainty

The largest uncertainty for me is how the application would behave under genuinely concurrent load or a node failure, because *I have not tested either*. The single-replica constraint (Section 2.3) is a direct, unavoidable consequence of the storage’s ReadWriteOnce access mode as configured, not a limitation I have separately verified is acceptable under real traffic. Reducing that uncertainty would mean either splitting the application into a storage-owning tier and a horizontally scalable stateless tier and load-testing the latter, or accepting the current single-replica design as appropriate only for a lab or a genuinely low-traffic internal tool.

5.4 Strategic Implications and Recommendation

I judge the current configuration appropriate as-is for continued use as a single-operator lab and reference implementation. Before I would reuse this pattern for a team or a production workload, I would make three changes, each named explicitly rather than left implicit:

- **split the storage-owning pod from a separately scalable stateless tier**, removing the single-replica constraint from anything that does not itself need direct volume access;
- **migrate state** to a remote OCI Object Storage backend, the same recommendation I carried over unresolved from Week 2; and
- **add an Ingress controller with TLS termination** in front of the current bare HTTP load balancer.

I carry these forward as concrete actions in Section 6.3.

5.5 Confidence Level and Final Judgement

My confidence in the findings of Chapter 4 is **high**: I backed every claim about the deployed environment with a live Terraform output, a `kubectl` command’s real output, or a browser request against the actual running system, following the evidentiary basis I set out in Section 2.6, including the two real bugs, which I diagnosed from their actual error output rather than inferred. My confidence in the forward-looking risk assessment is **medium**: the risks I

identify in Section 4.7 follow directly from the configuration as written and from mechanisms I understand, the RWO access mode, the local unlocked state backend, the worker image catalog's rotation, but I have *not* observed their real-world impact under conditions this lab did not exercise, concurrent multi-operator use, sustained load, or a worker image catalog structured differently from the two I have actually seen.

Chapter 6

Recommendations and Next Steps

6.1 Further Testing Needed

Two follow-up exercises would materially reduce the uncertainties I named in Section 5.3:

- a deliberate **node-failure drill**: cordon and drain the node currently holding the application's PersistentVolumeClaim and observe whether, and how quickly, the pod and its volume recover on a different node; and
- a basic **load test** against the load balancer's public endpoint, to see how a single-replica backend behaves as concurrent request volume increases, and how close it comes to the configured 10 Mbps flexible-shape ceiling.

6.2 Data to Validate

I verified everything covered in Chapter 4 against the live environment at the time of writing. **The item most likely to go stale first is the worker-image selection logic's assumption about OCI's naming conventions**, since Section 4.7 already flags that the fallback branch depends on OCI continuing to publish images matching the current filter pattern. If I, or anyone else, reuse this configuration later, I would re-run `terraform plan` and inspect the `debug.available_worker_images` output directly rather than trust the specific image name quoted in this documentation.

6.3 Recommended Actions

Before I reuse this pattern for a team or a production workload, here is what I would do first:

1. **Split the application into a storage-owning tier and a stateless tier.** Move anything that does not need direct `PersistentVolumeClaim` access into a separate Deployment with its own, larger replica count, removing the RWO-driven single-replica constraint from the parts of the application that do not actually need it.
2. **Enable a remote state backend.** Migrate `terraform.tfstate` to an OCI Object Storage bucket with locking, the same unresolved recommendation from Week 2, before any second operator touches this configuration.
3. **Add an Ingress controller with TLS termination.** The current Service exposes bare HTTP on port 80. A production-facing version of this build would run an Ingress controller in front of it, or add a second load balancer listener on 443, with a managed certificate.
4. **Narrow `is_endpoint_public`'s exposure,** or set it to `false` entirely and reach the Kubernetes API through a bastion or VPN, once direct `kubectl` access from an arbitrary local machine is no longer a lab convenience worth keeping.
5. **Add a Horizontal Pod Autoscaler** to the stateless tier proposed in item 1, once it exists, scaling replica count by CPU or memory utilization rather than a fixed count.
6. **Add monitoring and alerting** beyond the VCN flow-log capability already wired into the subnet module, integrating OCI Monitoring metrics and alarms for the cluster and node pool, and application-level health signals for the workload itself.

6.4 Monitoring Indicators

Table 6.1: Indicators worth monitoring if this environment moves beyond a single-operator lab.

Indicator	Why It Matters	Review Frequency
Pod readiness and restart count in <code>week3-demo</code>	Directly shows whether the application tier is reachable and stable	Continuous, via the existing readiness and liveness probes
<code>PersistentVolumeClaim</code> status (Bound vs. Pending)	Confirms the CSI driver and <code>StorageClass</code> are still provisioning storage correctly	On every deployment change
<code>debug.used.version_tagged_pool</code> on every <code>terraform plan</code>	Confirms the worker-image selection logic is still finding a version-tagged image rather than silently falling back	Before every apply that touches the node pool
Load balancer bandwidth utilization vs. the 10 Mbps flexible ceiling	Signals when the bandwidth range needs to be raised	Before and after any load test

6.5 Open Questions

Three questions I would answer before treating this configuration as a template for other builds:

- whether OCI File Storage Service, accessed through its own CSI driver and offering `ReadWriteMany` semantics, would be a better default than a block volume for any workload that genuinely needs concurrent multi-node access;
- whether the `Recreate` rollout strategy's brief downtime window is acceptable for a real, user-facing version of this application, or whether the storage-owning tier needs its own blue-green deployment pattern instead; and
- at what point the worker-image selection logic's architecture and variant exclusion list, currently just `aarch64` and `GPU`, would need to grow to stay correct as OCI ships new node-image families.

Appendix

.1 Glossary

Table 2: Terms used throughout this documentation.

Term	Meaning
OKE	Oracle Kubernetes Engine (Container Engine for Kubernetes): OCI's managed Kubernetes service, operating the control plane on the operator's behalf.
Node pool	A managed, homogeneous group of worker compute instances attached to an OKE cluster.
CNI	Container Network Interface: the plugin interface deciding how pods get IP addresses; this build uses VCN-native mode, assigning pods real, routable VCN addresses.
CSI	Container Storage Interface: the plugin interface letting Kubernetes provision real cloud storage; the OCI Block Volume CSI driver is what turns a PersistentVolumeClaim into a real block volume.
Cloud Controller Manager (CCM)	The OKE-installed component that provisions a real OCI Load Balancer automatically in response to a Kubernetes Service of type: <code>LoadBalancer</code> .
CRI-O	The container runtime running on this build's OKE worker nodes; enforces strict short-name resolution for image references.
PersistentVolume (PV)	The real, provisioned storage a PersistentVolumeClaim is bound to.
PersistentVolumeClaim (PVC)	A namespaced request for storage matching certain criteria; what a pod spec actually references.
StorageClass	A template telling Kubernetes which storage driver to use and with what parameters when dynamically provisioning storage.
ReadWriteOnce (RWO)	An access mode meaning the underlying volume can be mounted read-write by only one node at a time; the mode OCI block volumes are constrained to.
Deployment	A Kubernetes controller managing a set of identical pod replicas, handling rollouts and self-healing.
ReplicaSet	The object a Deployment creates and manages to keep a specific number of pods matching one pod-template hash running.
initContainer	A container that runs to completion before a pod's regular containers start; used here to render a proof-of-life page onto the shared volume.
Service	A Kubernetes object providing a stable network identity in front of a dynamic set of pods, selected by label.
Downward API	The Kubernetes mechanism exposing a pod's own metadata to its containers as environment variables or mounted files.
Variable	A Terraform input, supplied externally (person, <code>tfvars</code> file, CI pipeline).
Local	A Terraform value computed once inside the configuration from other available values.

.2 Command Reference

Table 3: Terraform and Kubernetes commands referenced in this documentation, all run against `weekThree/` during my work on this lab.

Command	Used for
<code>terraform plan/apply/destroy</code>	Standard Terraform lifecycle, used identically to Weeks 1 and 2
<code>oci ce cluster create-kubeconfig</code>	Generate a working <code>kubeconfig</code> for the cluster, using the exec-plugin credential pattern described in Section 4.3
<code>oci setup</code>	Fix the advisory Windows file-permission warnings the OCI CLI emits against the config and key files
<code>repair-file-permissions</code>	
<code>kubectl get nodes</code>	Confirm worker nodes have joined the cluster and are Ready before applying any application manifest
<code>kubectl apply -f file</code>	Apply a single Kubernetes manifest, run in dependency order across all five files in <code>k8s/</code>
<code>kubectl get pods -n week3-demo</code>	Check pod status in the application namespace
<code>kubectl get pods -n week3-demo -w</code>	Watch pod status transitions live; the tool that surfaced the roll-out deadlock in Section 4.4.2
<code>kubectl describe pod name -n week3-demo</code>	Full diagnostic detail for one pod, including container state, reason, and events; the tool that surfaced both bugs in Section 4.4
<code>kubectl get pvc -n week3-demo</code>	Confirm the PersistentVolumeClaim reached Bound at the expected size
<code>kubectl get svc -n week3-demo</code>	Confirm the Service's external IP has been populated by the Cloud Controller Manager
<code>kubectl get all -n week3-demo</code>	Single-command summary of every namespaced object's final state
<code>kubectl delete -f file</code>	Delete a single Kubernetes manifest, run in the reverse-dependency order specified in Section 3.8, strictly before <code>terraform destroy</code>

References

This documentation covers one specific deployment I built and verified live, rather than a literature-based analysis. I did not cite external sources, third-party claims, or published works. Every factual claim is instead traceable to a live Terraform output, a `kubectl` command's real output, or a browser request I ran directly against the deployed environment, following the evidentiary basis I set out in Section 2.6. I kept `References/myref.bib` in the repository for template compatibility, but it intentionally contains no entries, so no citation keys appear in this documentation.